IOWA DOM DOIT · DATA ENGINEERING

WorkBook

Data Engineering on Google Cloud —
all activities & labs, in schedule order

Day 1 · Monday, October 5, 2026 — Landing your data in BigQuery

Day 2 · Tuesday, October 6, 2026 — Your target operating model: lakehouse & medallion

9:00 AM – 4:30 PM CT · virtual

Instructor: Imran Ahmad, PhD

7 hands-on blocks on Day 1 · 6 on Day 2 · 6 reference-card appendices

Table of Contents

How to Use This Workbook 3

Activities & Labs at a Glance 5

Day 1 — Landing Your Data in BigQuery 6

SCHEDULE Schedule 7

ACTIVITY 1.1 Console Orientation — Projects, Datasets, and Tables 9

LAB 1.2 Loading Data into BigQuery 12

LAB 1.3 Datastream — PostgreSQL → BigQuery (split lab) 15

ACTIVITY 1.4 Four Ways to Load (and the Format Race) — DIN §24–§27 19

WALKTHROUGH 1.5 Dataform Repo Tour — a Real Silver→Gold Repository 29

LAB 1.6 Create and Execute a SQL Workflow in Dataform 32

ACTIVITY 1.7 Schedule a Daily Overwriting Query — DIN §29 35

Day 2 — Lakehouse & Medallion 40

SCHEDULE Schedule 41

ACTIVITY 2.1 Recap Quiz — Which GCP Tool Replaces Which Piece of Your Stack? 43

LAB 2.2 Federated Query with BigQuery and AlloyDB 45

ACTIVITY 2.3 External Tables — Bronze Made Tangible (DIN §30) 48

LAB 2.4 Querying External Data and Iceberg Tables 52

DEEP-DIVE 2.5 Apache Iceberg, Explained 55

WORKSHOP 2.6 Capstone — Map One of Your Real Extract Jobs 58

Appendices — Reference Cards 62

APPENDIX A. Migration Map 63

APPENDIX B. T-SQL → BigQuery SQL 66

APPENDIX C. BigQuery Console Survival Guide 69

APPENDIX D. GCP for SQL Server Professionals 72

APPENDIX E. Power BI ↔ BigQuery Connector Guide 75

APPENDIX F. Glossary 79

How to Use This Workbook

These are the student-facing activity guides and lab companions for the two-day *Data Engineering on Google Cloud* course delivered for the Iowa DOM DoIT data team, gathered in schedule order. Each section opens with a banner showing **when** it runs (Central Time), **how long** it takes, and **what kind** of block it is:

- **Lab** — runs in the Google Skills classroom in a temporary, timed GCP project. The lab's own click-by-click steps appear in the classroom when you launch it; the section here is your orientation, gotcha list, and self-check around those steps.
- **Activity** — an instructor-led follow-along you do in your own lab project. Every step, command, and SQL statement is printed here in full.
- **Walkthrough / Deep-dive / Workshop** — guided reading, explainer, or group exercise.

Every script and SQL file the activities use is printed in full at the end of its activity, and the reference cards the activities point to are reproduced as appendices — no other file is needed.

Before Day 1 — the three things you need

1. **The Skills classroom:** https://www.skills.google/ilt/classrooms/37990 — sign in and launch each lab when it's called.
2. **The Do-It-Now (DIN) page:** https://roitraining.github.io/gcp-demos/bigquery.html — short BigQuery exercises referenced by section number (§). The DIN activities used in class are reproduced in this workbook.
3. **Your `class` dataset** — created by you in Activity 1.1 (Day 1, 10:10). Every loading activity writes into it. Don't skip it.

How the labs work

- Labs run in the Google Skills classroom. Open the classroom, sign in, and launch the lab when it's called.
- Each lab gives you a **temporary GCP project** with real credentials. When you start a lab, **the lab timer starts** — watch it. When time expires, the project is deleted. Nothing in a lab project is permanent, and that's by design: it's a disposable sandbox.
- Lab projects are safe to break. You cannot damage anything that matters.
- The lab's own step-by-step instructions live in the classroom. The companions in this pack add orientation, gotchas, and context around them — they don't replace the lab instructions.
- One thing you create yourself (not in a lab): on Day 1 morning you create a dataset named `class` in your lab project. Every afternoon loading activity writes into it — don't skip that step.

Ground rules

- **Ask anything.** You're senior engineers who are new to one console. "Where do I click?" questions are expected and welcome — sixteen people learning one new UI goes faster when nobody sits stuck.

- **Parking lot.** Questions that would pull the room off the day's path get written down visibly and answered before the day ends.
- **Labs are safe to break.** Temporary projects, free or pennies of cost, nothing connected to state systems. If something fails, that's a learning moment — say so and we'll look at it together.
- **This pack is yours.** Take it with you. The reference cards were written to be useful at your desk the week after class, not just during it.

Activities & Labs at a Glance

Every hands-on block of the two days, with its time slot (Central Time). Click a title to jump to it.

Day	Time (CT)	Length	Type	Block	Page
D1	10:10–10:25	15 min	Activity	1.1 Console Orientation — Projects, Datasets, and Tables	9
D1	10:40–11:10	30 min	Lab	1.2 Loading Data into BigQuery	12
D1	11:50–12:00 (Part 1) · 13:00–13:15 (Part 2)	10 + 15 min	Lab	1.3 Datastream — PostgreSQL → BigQuery (split lab)	15
D1	13:45–14:30	45 min	Activity	1.4 Four Ways to Load (and the Format Race) — DIN §24–§27	19
D1	15:10–15:25 (inside the 14:45–15:25 ELT & Dataform block)	15 min	Walkthrough	1.5 Dataform Repo Tour — a Real Silver→Gold Repository	29
D1	15:25–16:05	40 min	Lab	1.6 Create and Execute a SQL Workflow in Dataform	32
D1	16:20–16:30	10 min	Activity	1.7 Schedule a Daily Overwriting Query — DIN §29	35
D2	09:00–09:15	15 min	Activity	2.1 Recap Quiz — Which GCP Tool Replaces Which Piece of Your Stack?	43
D2	10:45–11:15	30 min	Lab	2.2 Federated Query with BigQuery and AlloyDB	45
D2	11:15–11:35	20 min	Activity	2.3 External Tables — Bronze Made Tangible (DIN §30)	48
D2	13:00–13:45	45 min	Lab	2.4 Querying External Data and Iceberg Tables	52
D2	13:45–14:00	15 min	Deep-dive	2.5 Apache Iceberg, Explained	55
D2	15:20–15:50	30 min	Workshop	2.6 Capstone — Map One of Your Real Extract Jobs	58

DAY 1 · MONDAY, OCTOBER 5, 2026

Day 1 — Landing Your Data in BigQuery

Console confidence and getting data in: loading files, streaming database changes live, and your first Dataform pipeline.

Day 1 Schedule

"Landing your data in BigQuery" · 9:00 AM–4:30 PM CT · virtual. Day 1 is about console confidence and getting data in: loading files, streaming database changes live, and your first Dataform pipeline.

Time	Block	What you'll walk away with
09:00–09:25	**Welcome & your migration story**	The shape of the two days: kill the extracts → land in BigQuery → Bronze/Silver/Gold → your existing Power BI reports
09:25–10:10	**DE building blocks on GCP**	The GCP moving parts — projects, datasets, Cloud Storage, BigQuery — mapped to the sources/transforms/sinks model you already think in
10:10–10:25	**Hands-on: find your way around the console**	Three starred projects (`bigquery-samples`, `bigquery-public-data`, `roi-bq-demos`) and your own `class` dataset, created by you (DIN §3 — see note below)
10:25–10:40	**Break**	
10:40–11:10	**Lab: your first BigQuery load** — *Loading Data into BigQuery*	Your first finished lab in an unfamiliar console: a real table, loaded by you
11:10–11:50	**Killing the nightly extracts**	Why log-based change data capture (Datastream) replaces the extract window — SQL Server is a supported source, with no triggers and no load on production
11:50–12:00	**Lab: set up a live replication stream (part 1)** — *Datastream: PostgreSQL → BigQuery*	A running stream replicating a database into BigQuery before lunch
12:00–13:00	**Lunch** — your stream keeps replicating	
13:00–13:15	**Lab part 2: watch changes land**	The CDC moment, felt: rows changed at the source appear in BigQuery with no job run
13:15–13:45	**Loading files the right way**	The extract-and-load pattern for file drops: console loads, the `bq` CLI, scheduled ingest with the BigQuery Data Transfer Service, and when an external table beats a load
13:45–14:30	**Hands-on: four ways to load (and the format race)**	Console & CLI load, the autodetect N/A pitfall, the 60-million-row CSV-vs-Parquet race, and `LOAD DATA` SQL (DIN §24–§27) — plus the rule: pin schemas for anything recurring
14:30–14:45	**Break**	
14:45–15:25	**ELT & Dataform — your Silver→Gold tool**	How Dataform turns your SQL into a versioned, tested, scheduled pipeline — the tool your own architecture doc already names

Time	Block	What you'll walk away with
15:25–16:05	**Lab: build a SQL workflow in Dataform** — *Create & Execute a SQL Workflow in Dataform*	A working workflow you built yourself: models, dependencies, assertions
16:05–16:20	**When SQL isn't enough + automation**	The escape hatches (Pub/Sub, Dataflow) and the automation map: Cloud Scheduler ≈ SQL Agent, Composer ≈ the orchestrator, Dataform release configs ≈ your CI/CD
16:20–16:30	**Wrap-up: schedule your first query**	Your first piece of BigQuery automation: a query on a daily schedule overwriting a table (DIN §29, step 5) — no SQL Agent box required

Worth knowing before you arrive

- **Every hands-on block above links to its section in this workbook.** Read the section before the block starts — it carries the gotchas.
- **The `class` dataset matters.** You create it yourself at 10:10, in your own lab project. Every loading activity this afternoon writes into it — it's the one step that can't be skipped or made up later.
- **Labs live in the Google Skills classroom.** Launching a lab starts its timer and hands you a temporary GCP project. The lab's own steps are in the classroom; the companions in this folder add orientation and gotchas.
- **"DIN" = the Do-It-Now activities page:** https://roitraining.github.io/gcp-demos/bigquery.html — a public set of short BigQuery exercises referenced by section number (§) all day. All four loading DINs this afternoon are free.
- **The Datastream lab uses PostgreSQL** because it's free-tier friendly. The mechanics — connection, stream, backfill, change capture — are identical for SQL Server, which is a supported Datastream source (docs).
- **Demo data you'll touch today:** the public `roi-bq-demos` and `bigquery-public-data` projects, and the public GCS bucket `jwd-gcp-demos`.

Reference cards to keep open today

- BigQuery console survival guide — during the 10:10 console orientation and every lab
- T-SQL → BigQuery SQL — the moment you start typing queries
- GCP for SQL Server professionals — the mapping card behind every block today

1.1 Activity: Console Orientation — Projects, Datasets, and Tables

ACTIVITY **Day 1 · Mon Oct 5, 2026** Time: **10:10–10:25 CT** Length: **15 min**

Guided follow-along on Day 1 (10:10–10:25 AM CT), right after M1: your first hands-on tour of the BigQuery console. Based on §3 ("Exploring projects, datasets, and tables") of the Do-It-Now (DIN) activities page. No cost; everything happens in the console.

What you'll do

- Open the BigQuery console and pin it for the day
- Star three projects into your Explorer: `bigquery-samples`, `bigquery-public-data`, `roi-bq-demos`
- Explore a public dataset and a 100-billion-row table (schema, details, preview — no query, no cost)
- Create your own dataset named `class` — **required** for this afternoon's loading activities

Why it matters for your migration

- **Project → dataset → table** is the GCP resource hierarchy — the same shape as your **instance → database → schema.table**. Everything you build on GCP hangs off this skeleton.
- **Starring is your first taste of "query data where it lives."** Starred projects belong to someone else and stay read-only, yet you can query across them without copying a byte. That is the mechanic behind BigQuery as a single source of truth.
- **The `class` dataset is your landing zone.** Every load this afternoon lands there — think of it as your Bronze layer in miniature.
- **The 100-billion-row table is the scale story, felt.** You're about to browse 100B rows with zero servers provisioned.

Before you start

- You are signed in to the Google Cloud console with the lab user account from your Skills classroom lab.
- You know how to check which project is selected: the **project picker** in the top bar.

Steps

Follow your instructor's screen; the annotations below tell you what to notice at each stop.

1. Get to BigQuery

1. Open the Google Cloud console.
2. Open the navigation menu (the "hamburger" icon, top left) and choose **BigQuery** — or type "BigQuery" into the search bar at the top.
3. Hover over **BigQuery** in the navigation menu and click the pin, so it stays at the top all day.
4. In the **Explorer** pane (left), expand your project. Notice it's empty — nothing exists yet.

2. Star three projects

Starring pins someone else's project into your Explorer so you can browse and query it.

5. In the Explorer pane, click **+ Add**, then choose **Star a project by name**.
6. Type the name exactly and confirm: `bigquery-samples`
7. Repeat for: `bigquery-public-data`
8. Repeat for: `roi-bq-demos`
9. Star any additional projects your instructor calls out.

(Console docs: starring a project)

3. Explore datasets

10. Expand **bigquery-samples** in the Explorer. Each entry under it is a **dataset** — a collection of tables and views that is secured as a unit and lives in a specific location.
11. Click **wikipedia_benchmark** and read the metadata shown in the right pane.
12. Click the three-dot menu (⋮) next to a dataset name and note the available actions. **This menu is where `Create table` lives** — you'll use it at 10:40.

4. Create your `class` dataset — required

13. Click the ⋮ menu next to **your own project** → **Create dataset**.
14. Dataset ID: `class` — exactly that, lowercase, no spaces. The other defaults are fine for class; note the **location** shown before you confirm.
15. Click **Create dataset**, then expand your project to confirm it appears.

Do not skip this step. The afternoon loading activities (DIN §24–§27) write into your `class` dataset. If it doesn't exist by 13:45, you'll be creating it while everyone else is loading data.

5. Explore tables

16. Expand **bigquery-samples** → **wikipedia_benchmark** and review its tables.
17. Click the **100B** table — yes, that's 100 billion rows.
18. **Schema** tab: column names and types — the same idea as the SSMS columns view or `sp_help`.
19. **Details** tab: storage size, row count, last-modified time.
20. **Preview** tab: sample rows *without running a query* — free browsing of a 100-billion-row table.
21. Open the ⋮ menu next to the table name and note the table-level actions.

Common gotchas

> **Wrong project selected.** The console remembers your last-used project and happily shows it to you. If you "can't see what the instructor sees," check the **project picker in the top bar first**. This is the number-one cause of confusion today.
>
> **Starred projects are read-only.** You cannot create anything inside `bigquery-samples`, `bigquery-public-data`, or `roi-bq-demos` — permission errors there are expected. Create things in **your** project.
>
> **Dataset location is permanent.** You choose it at creation and cannot change it afterward — fixing it means recreating or copying the dataset. For class, the default is fine; for your real warehouse, it's an architecture decision.
>
> **The three-dot (⋮) menus hide the actions.** Create dataset, Create table, Delete — if you can't find a button, look for the ⋮.
>
> **New dataset not showing?** Expand your project node in the Explorer pane, or click the refresh icon at the top of the pane.

If you finish early

- Browse into `bigquery-public-data`, pick any dataset that looks interesting, and open a table's **Preview** tab.
- On the 100B table's **Details** tab, find the exact row count and total size.
- Open your own `class` dataset's ⋮ menu and locate **Create table** — your next stop at 10:40.

Self-check

- [ ] Three starred projects appear in your Explorer: `bigquery-samples`, `bigquery-public-data`, `roi-bq-demos`
- [ ] Your own project contains a dataset named exactly `class`
- [ ] You can open the Schema, Details, and Preview tabs on `bigquery-samples.wikipedia_benchmark.100B`
- [ ] You know where the project picker is, and which project is yours
- [ ] You found **Create table** in your dataset's ⋮ menu

What this replaces in your current stack

You do this today	On GCP
SSMS Object Explorer: server → databases → tables	BigQuery Explorer pane: projects → datasets → tables
Linked servers / browsing another instance	Starred projects — query across projects without copying data
`CREATE DATABASE` plus file and filegroup sizing	Create dataset — one dialog, nothing to size

Next: Lab 1 companion — Loading Data into BigQuery

1.2 Lab: Loading Data into BigQuery

LAB **Day 1 · Mon Oct 5, 2026** Time: **10:40–11:10 CT** Length: **30 min**

Companion notes for the first lab of Day 1 (10:40–11:10 AM CT). The lab's own step-by-step instructions live in the Google Skills classroom (Course 1, Module 1 lab) — this page adds orientation and gotchas around them, not a second copy of the steps.

What you'll do

- Load data into BigQuery from various sources
- Load using both the Google Cloud console and the `bq` command-line tool
- Use DDL to create tables

Why it matters for your migration

This is the simplest possible version of what your nightly extract jobs do: a file becomes a queryable table. It's the Bronze-landing mechanic for file-based sources, and it's deliberately easy — your first win in an unfamiliar console, not a skills test. By 13:45 you'll have loaded data four more ways, each one closer to how your real migration will run.

Before you start

- [ ] The lab is launched from the Skills classroom and the **lab timer is running** — having the instructions open is not the same as having the lab started.
- [ ] You're signed in to the console with the **lab credentials** the classroom gave you, not a personal or organizational account.
- [ ] The **project picker** in the top bar shows your lab project.
- [ ] Your `class` dataset exists (you created it in the 10:10 console orientation). This lab doesn't strictly require it, but the 13:45 loading block does — if it's missing, create it now: your project's ⋮ menu → **Create dataset** → ID `class`.

Common gotchas

> **The autodetect-schema checkbox.** Miss it — or feed it a file it can't make sense of — and BigQuery builds a table with a single `string` column holding whole rows. If that happens to you: note it and move on; don't spend lab time fixing it. At 13:45 we reproduce this failure on purpose, and you'll see exactly why it happens and how to load around it.
>
> **"Enable API" prompts.** Normal on first use of a service in a fresh lab project. Click **Enable** and keep moving — don't stop to read the API catalog page.
>
> **Cloud Shell's first open.** Expect a few consent / authorize / continue clicks the first time you launch it. One-time per project; later opens are instant.
>
> **Wrong project.** The console defaults to the last-used project. If a menu item "doesn't exist," check the picker before anything else.
>
> **Can't find BigQuery?** Navigation (hamburger) menu → **BigQuery** — hover and pin it — or use the search bar.

If you finish early

Finishing in 10–12 minutes is expected for people who write SQL all day. Use the extra time on **§24 of the Do-It-Now (DIN) activities page**: load a CSV from the public `jwd-gcp-demos` Cloud Storage bucket into **your own `class` dataset** using the console. That's exactly what the whole group does at 13:45 — you'll be the resident expert.

Note your load time. It feeds the CSV-vs-Parquet race this afternoon (DIN §26).

Debrief: three ways in, same destination

1. **Console click-through** — point, click, table. The GUI your one-off imports always wanted.
2. `bq load` — the same job from the command line: scriptable, repeatable, the shape of automation.
3. **DDL (`CREATE TABLE` / `LOAD DATA`)** — load logic as code, where it can live in source control and ride your CI/CD discipline.

Your SSIS packages are the fourth way in. Datastream — next hour — is the fifth, and it's the one that kills the nightly extract entirely.

Self-check

- [ ] You loaded at least one table through the console
- [ ] You ran (or watched run) a `bq load` from Cloud Shell
- [ ] You created a table with DDL
- [ ] Your `class` dataset still exists — and if you did the early-finisher activity, it now holds a table and you've written down the load time

What this replaces in your current stack

You do this today	On GCP
SQL Server Import and Export Wizard / one-off flat-file SSIS package	Console **Create table** load
BCP / `BULK INSERT` scripts	`bq load` from Cloud Shell
DDL deployment scripts in your CI/CD repo	`CREATE TABLE` / `LOAD DATA` DDL (GoogleSQL)

1.3 Lab: Datastream — PostgreSQL → BigQuery (split lab)

LAB **Day 1 · Mon Oct 5, 2026** Time: **11:50–12:00 (Part 1) · 13:00–13:15 (Part 2) CT** Length: **10 + 15 min**

Your companion for the Day 1 (Mon Oct 5, 2026) Datastream lab, which runs in two halves around lunch: **Part 1, 11:50–12:00 CT** (set up and start the stream) and **Part 2, 13:00–13:15 CT** (validate the backfill, then watch a live change land). The lab's own step-by-step instructions live in the Google Skills classroom (course 1, module 2) — this companion adds orientation, context, and the things that commonly go wrong. It follows the Module 2 session on replication and migration.

What you'll do

- Prepare a Cloud SQL for PostgreSQL instance using the Google Cloud console.
- Import data into the Cloud SQL instance.
- Create a Datastream connection profile for the PostgreSQL database (the source).
- Create a Datastream connection profile for the BigQuery destination.
- Create a Datastream stream and start replication.
- Validate that the existing data — and every change you make afterwards — replicates correctly into BigQuery.

Why it matters for your migration

This lab is your project's pilot wearing a lab costume. The thing your legacy extracts do tonight — copy what changed in the database into the warehouse — this stream does continuously, with no batch window. Part 2 is deliberately scheduled after lunch so you experience the headline:

> **Your data changed while you ate lunch. No nightly job ran, no extract window opened, no SSIS package executed.**

If you take one feeling back to work from Day 1, make it that one.

> **Why PostgreSQL and not SQL Server?** PostgreSQL is in this lab because it's free-tier friendly — the mechanics are **identical for SQL Server, which IS a supported Datastream source**. Connection profiles, streams, backfill, CDC: everything you click today maps one-to-one onto your SQL Server migration. (Details: Configure a source SQL Server database for Datastream.)

Before you start

- [ ] Your lab is launched from the Skills classroom and you can see your **lab-provided GCP project** and credentials on the lab panel. You work in that project — not any other project you may see in the console's project picker.
- [ ] Part 1 is only 10 minutes. Have the lab open and be ready to move when the block starts; the instructor will drive the setup briskly and you'll follow click-for-click.

☐ Nothing to install — the console does everything. The lab itself prepares the PostgreSQL source for replication; that prep is one of the objectives, so notice what it configures. That is the part your M&O team will ask about later.

Part 1 — before lunch (~10 min): get the stream running

Follow the lab's own steps. Here is the shape of what you're doing and how to think about each piece:

1. **Prepare and seed the source.** You stand up a Cloud SQL for PostgreSQL instance and import data into it. This instance plays the role of your SQL Server warehouse: the database that never stops changing.
2. **Create two connection profiles.** A connection profile is a saved, tested connection definition — the same idea as an SSIS connection manager. One points at the source (PostgreSQL), one at the destination (BigQuery).
3. **Create the stream.** You pick the two profiles, choose what to replicate, and configure how the existing data (backfill) and future changes (CDC) will flow.
4. **Start the stream — and leave it running.** The stream begins its backfill. Once it shows it's up, Part 1 is done.

Then go to lunch. **Do not pause, stop, or clean up anything** — the stream replicating while you eat *is* the lesson.

Part 2 — after lunch (~15 min): the CDC moment

1. **Validate the backfill.** In the BigQuery console, find the tables the stream created in your dataset. Compare row counts against the source and eyeball the data. Notice the extra metadata columns Datastream adds to each row — including `change_type` and `is_deleted`, which tell you what happened to the row.
2. **Make a source change.** Run the lab's UPDATE/INSERT against the Cloud SQL instance.
3. **Watch it land.** Keep an eye on the stream's status, and refresh your BigQuery view until the change appears — typically within about a minute of the source commit. No job ran; nothing was scheduled. That is what "kill the nightly extracts" means in practice.

Common gotchas

"Where are my BigQuery tables?" Destination tables appear *as the stream runs*. If you go looking before the backfill has started creating them, you'll think it's broken. Give it time, then refresh the Explorer panel.

Region mismatch. The connection profiles' region must match the Cloud SQL instance and the BigQuery dataset location. A mismatched region is the classic first-timer failure in this lab — if a connection test or a stream fails early, check regions first.

Where to watch stream status. Status lives on the **Datastream** → **Streams** page in the console. Streams take a little time to come up (Starting → Running); that startup latency is normal, not an error.

Wrong project in the picker. The console remembers projects from other sessions. Before you create anything, confirm the project picker at the top shows your lab project — otherwise your instance, profiles, or stream land somewhere you can't see them.

API-enable prompts. First use of Datastream or Cloud SQL in a fresh project can pop "API not enabled" prompts. Click **Enable** and continue; it happens once per project.

"My change hasn't shown up in BigQuery." After-lunch changes usually land within a minute or so. If yours hasn't: refresh the table preview, then check the stream status page before assuming failure.

Stream won't start and the clock says lunch? Don't burn the lunch window debugging. Watch the after-lunch validation live — the moment lands the same on a shared screen — and re-run the lab yourself in the evening if you want the clicks.

Cost awareness (for real life, not today). Today's resources live in a time-boxed lab project. In your own environment, a running stream and a Cloud SQL instance bill while they're up — plan for them as always-on resources, not batch jobs.

If you finish early

- **Part 1:** open your stream's details page and watch the status and throughput as the backfill begins. Compare with a neighbor whose stream started at a different moment.
- **Part 2:** make a second change of a different kind — a DELETE — and find how the metadata columns (`change_type`, `is_deleted`) represent it in BigQuery.

Self-check

Part 1, before you leave for lunch:

- [ ] Both connection profiles exist — one for the PostgreSQL source, one for the BigQuery destination.
- [ ] The stream is created and its status shows it starting or running.
- [ ] You have paused, stopped, and deleted nothing; everything stays up over lunch.

Part 2:

- [ ] You found the tables Datastream created in BigQuery, and their row counts match the source.

- [ ] You can point at the metadata columns (`change_type`, `is_deleted`) on a replicated row.
- [ ] Your own UPDATE/INSERT from the source appeared in BigQuery without you running any load job.

What this replaces in your current stack

- **The nightly extract window** → an always-on stream; freshness stops depending on a schedule.
- **The SQL Agent job + SSIS extract package** → Datastream backfill + CDC; there is no package to deploy or retry.
- **Hand-rolled delta detection** (timestamp columns, triggers, checksums) → the database change log, which already records every change.

1.4 Activity: Four Ways to Load (and the Format Race) — DIN §24–§27

ACTIVITY **Day 1 · Mon Oct 5, 2026** Time: **13:45–14:30 CT** Length: **45 min**

Your guide for the Day 1 (Mon Oct 5, 2026) afternoon loading block, 13:45–14:30 CT. The instructor demonstrates each activity, then you do it yourself. All four activities come from the Do-It-Now (DIN) activities page (§24–§27), and all four are free — batch loads cost nothing. The concepts behind them are in the Module 3 notes.

What you'll do

Four short activities, in order:

1. **§24 — Console & CLI load.** Load Monday's 1M-row vendor file with the console, append Tuesday's with the `bq` CLI.
2. **§25 — The autodetect pitfall.** Watch schema autodetect fail two different ways on dirty vendor data — once loudly, once silently — then fix it with a pinned schema.
3. **§26 — CSV vs Parquet race.** Load the same 60 million rows twice: once as one gzipped CSV (~2 minutes), once as Parquet (seconds).
4. **§27 — LOAD DATA in SQL.** Do the activity-1 loads again as pure SQL statements with an explicit schema.

Why it matters for your migration

This is your Bronze-landing pattern, hands-on: vendor file → Cloud Storage bucket → BigQuery table — the same shape as the legacy extract jobs you're replacing. Along the way you'll pick up the four rules that keep a Bronze layer honest: pinned schemas, the right file formats, free batch loads instead of row-by-row inserts, and everything expressible as repeatable SQL that lives in source control and a scheduler.

Before you start

- ☐ You created a dataset named `class` in this morning's exploring-datasets activity (DIN §3). Everything in this block loads into `class`. If you don't have it, create it now: BigQuery console → your project → Create dataset.
- ☐ The **BigQuery console** is open, and the project picker at the top shows **your lab project**.
- ☐ You know how to open **Cloud Shell** (the terminal icon at the top right of the console).

The two asset files you need are printed in full at the end of this activity: `autodetect_demo.sh` (activity 2) and `load_data_example.sql` (activity 4). They are byte-identical copies of the DIN page's source files (shell source, SQL source), so you can work from the local copies even if the network is slow.

Common gotchas — the whole block

- **Wrong project picker.** If the console shows a different project than your lab project, your tables land where you can't find them. Check the picker first, every time.
- **Cloud Shell first-run clicks.** The first time you open Cloud Shell you may get consent/authorize screens — click through them (Continue/Authorize) and wait for the shell prompt.
- **API-enable prompts.** A fresh lab project sometimes asks to enable an API the first time you touch a service. Click **Enable** and carry on.
- **Cloud Shell is scratch space.** Files you create there (activity 2) live on a temporary VM, not in your bucket. That's all these activities need — just don't expect them to survive the session.

Activity 1 (DIN §24) — Loading data: console and CLI

What you'll do. A vendor drops a usage file into a Cloud Storage bucket every day. You'll load Monday's file (`usage_2026-07-20.csv`, 1,000,000 rows) into a new table with the console, then append Tuesday's file with the CLI.

Why it matters. The console form and the `bq` command are two front ends over the same job API — the thing you'll script later is the thing you click today. This is the extract-job replacement in miniature.

Steps.

1. In the **Explorer** panel, click the three-dot menu next to your `class` dataset and select **Create table**.
2. Set **Create table from** to **Google Cloud Storage**.
3. In the file field, enter `jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv` — **without** the `gs://` prefix (the console adds it).
4. Confirm **File format** is **CSV**. Name the table `usage`.
5. In the **Schema** section, check **Auto detect**, click **Create table**, and wait for the load to finish.
6. Open the new `usage` table's **Details** tab: **Number of rows** is **1,000,000**. Check the **Schema** tab: `minutes` came out **INTEGER** and `event_date` came out **DATE** — autodetect chose well *this time*. Remember that phrase.
7. Tuesday's file has arrived. Open **Cloud Shell** and append it to the same table:

```
bq load --autodetect --source_format=CSV \
  class.usage \
  gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-21.csv
```

8. Back in the console, refresh the **Details** tab: **2,000,000** rows. Loads append by default; `--replace` would have truncated the table first.
9. Click **View all jobs** and look at the two most recent jobs: both are **Load** jobs, identical in kind. Console and CLI, same machinery.

Expected result. Table `class.usage` holds 2,000,000 rows, the job history shows two Load jobs, and nothing was billed — batch loads are free, and reading a bucket owned by another project is fine (you only need read access to the objects).

> **Gotcha box**
>
> - `gs://` **in the console file field.** The console wants `bucket/path` with no `gs://` prefix; pasting the full URI is the most common way this step "can't find the file."
> - **Cloud Shell pointing at another project.** `bq` acts on whichever project Cloud Shell is configured for. If `class` "doesn't exist," check the project before re-creating anything.

Activity 2 (DIN §25) — Schemas: autodetect is a starting point (the pitfall)

What you'll do. Generate two dirty vendor files locally, load both with autodetect, and watch the same bad value cause a **loud failure** in one and a **silent failure** in the other. Then pin a schema, declare the vendor's null marker, and load cleanly.

Why it matters. Your migration will ingest vendor and partner files with exactly this class of dirt. The fix pattern — a pinned schema JSON in source control plus `--null_marker` — slots straight into your CI/CD discipline: the schema becomes a versioned artifact, not a console setting. Autodetect is the "Suggest Types" button: fine for a first look, never for a recurring pipeline.

Steps.

1. Open the local copy of `autodetect_demo.sh`, copy its entire contents, and paste them into your Cloud Shell terminal. It creates three files:
   - `daily_usage_mon.csv` — Monday's big drop (100,000 rows); the vendor writes `N/A` in the `minutes` column for meetings that never started; Monday's `N/A` sits at row 90,000
   - `daily_usage_tue.csv` — Tuesday's small drop (1,000 rows); its `N/A` is at row 700
   - `daily_usage_schema.json` — the pinned schema used by the fixed loads
2. Look at the first rows of Monday's file, and find the `N/A` rows in both files:

```
head -3 daily_usage_mon.csv
grep -n "N/A" daily_usage_mon.csv daily_usage_tue.csv
```

3. **The loud failure.** Load Monday's file, letting autodetect choose the types:

```
bq load --autodetect --source_format=CSV class.daily_usage daily_usage_mon.csv
```

The load **fails**. Read the error: "Unable to parse." Autodetect sampled the start of the file, saw only numbers in `minutes`, chose INTEGER — and row 90,000 said `N/A`.

4. **The silent failure.** Load Tuesday's file the same way:

```
bq load --autodetect --source_format=CSV class.daily_usage daily_usage_tue.csv
```

This load **succeeds**. Check what autodetect decided:

```
bq show --schema class.daily_usage
```

`minutes` is **STRING**. Tuesday's file is small enough that the `N/A` fell *inside* autodetect's sample, so it picked the only type that fits everything. Now try to use the data in the BigQuery editor:

```
SELECT SUM(minutes) AS total_minutes FROM class.daily_usage
```

The query fails: no version of SUM accepts a STRING. Same bad value, two different outcomes, decided only by where the value sat relative to the sample. Monday failed loudly — annoying, but you found out immediately. Tuesday failed silently — the load worked, the schema is wrong, and the damage surfaces later in every query that does math on `minutes`. **The silent one is the one that costs you days.**

5. **Pin the schema, name the marker.** Look at the schema file, then reload both days correctly — `--replace` on the first load (clear out Tuesday's silent-failure table), plain append on the second:

```
cat daily_usage_schema.json
```

```
bq load --replace --source_format=CSV \
  --schema=daily_usage_schema.json \
  --skip_leading_rows=1 \
  --null_marker='N/A' \
  class.daily_usage daily_usage_mon.csv
```

```
bq load --source_format=CSV \
  --schema=daily_usage_schema.json \
  --skip_leading_rows=1 \
  --null_marker='N/A' \
  class.daily_usage daily_usage_tue.csv
```

6. Verify in the BigQuery editor:

```
SELECT
  COUNT(*) AS rows_loaded,
  COUNTIF(minutes IS NULL) AS null_minutes,
  SUM(minutes) AS total_minutes
FROM class.daily_usage
```

Expected result. 101,000 rows loaded, exactly **2** NULL minutes (the meetings that never started), and SUM works.

Gotcha box

- **The Monday failure is the lesson, not a mistake.** If your load "breaks," you did it right — read the error message before moving on.
- **Don't drop `--skip_leading_rows=1` once the schema is pinned.** Autodetect was quietly skipping the header for you; with an explicit schema you must say so yourself, or the header row fails against your INTEGER column.
- **Re-running appends.** Run the Tuesday load twice and you'll have 102,000 rows. Loads append by default — `--replace` exists for a reason.
- **Cheap path to a schema file.** Let autodetect run once on a known-good file, dump it with `bq show --schema`, review and edit, then load with `--schema` from then on.

Activity 3 (DIN §26) — Load performance: file formats matter (CSV vs Parquet race)

What you'll do. Sixty million rows of usage history sit in the demo bucket twice — once as a single gzipped CSV (~317 MiB), once as 100 Parquet files (~829 MiB total). You'll load both and compare the times.

Why it matters. This feeds directly into your Bronze-layer decision: what format do landing files arrive in, and what do you convert to? Wherever you control the format, land Parquet (or better). For vendor CSVs you *don't* control, the mitigation is many smaller uncompressed files over one large gzip. And notice the schema angle: Parquet carries real types in the file — the whole `N/A`-in-a-numeric-column failure class disappears when the format can't represent it.

Steps.

1. Look at both stagings, then place your bet — which loads faster?

```
gcloud storage ls -l gs://jwd-gcp-demos/ingest_demo/big/
gcloud storage du -s --readable-sizes gs://jwd-gcp-demos/ingest_demo/big_parquet
```

2. Load the gzipped CSV, with an explicit schema, and keep an eye on the elapsed-seconds counter `bq` prints while it waits:

```
bq load --source_format=CSV \
  --skip_leading_rows=1 \
  --schema=meeting_id:STRING,user_id:STRING,minutes:INTEGER,event_date:DATE \
  class.usage_history_csv \
  gs://jwd-gcp-demos/ingest_demo/big/usage_big.csv.gz
```

3. Around **two minutes** later it finishes. While you waited: a gzip file **cannot be split**, so one worker had to decompress and parse it from start to finish, alone. The 100 Parquet files can be handed to 100 workers at once, and Parquet is a binary columnar format, so there is no text parsing at all. That's how "more than twice the bytes" loads roughly 20x faster.
4. Load the Parquet copy of the same data. No `--schema` this time — Parquet files carry their own:

```
bq load --source_format=PARQUET \
  class.usage_history_pq \
  'gs://jwd-gcp-demos/ingest_demo/big_parquet/usage_big-*.parquet'
```

5. Confirm both tables:

```
bq show class.usage_history_csv
bq show class.usage_history_pq
```

Expected result. CSV load: about two minutes. Parquet load: seconds, despite being over twice the bytes. Both tables show **60,000,000** rows, and the Parquet-loaded table got the right types (INTEGER, DATE) from the files themselves.

Gotcha box

- **It is not hung.** Two minutes of waiting on the CSV load *is* the demonstration. Don't Ctrl-C — watch the elapsed counter and think about what one lonely worker is doing.
- **Quote the wildcard URI.** The single quotes around the Parquet path stop your shell from trying to expand the `*` itself — BigQuery expands it.
- **Don't pass `--schema` with Parquet.** The files know their own types; an inline schema is unnecessary noise.

Activity 4 (DIN §27) — LOAD DATA: loading with SQL

What you'll do. Re-do the activity-1 loads as pure SQL statements from `load_data_example.sql`, with the schema pinned in the column list.

Why it matters. A load expressed as SQL can live inside a script, a stored procedure, or a scheduled query — it hooks straight into the automation instincts you already have. One scheduled `LOAD DATA` with a date-built URI is the nightly-extract replacement in a single statement; the Day-1 closing demo schedules a query in one click, and you'll recognize the shape.

Steps.

1. Open `load_data_example.sql` and review the `-- load monday query` statement: the column list pins the schema the way `--schema` did on the CLI, and the `FROM FILES` options mirror the `bq load` flags. Run it in the BigQuery query editor. It finishes in a few seconds and creates the `usage_sql` table.

```
LOAD DATA INTO class.usage_sql (
  meeting_id STRING,
  user_id STRING,
  minutes INT64,
  event_date DATE
)
FROM FILES (
  format = 'CSV',
```

```
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv']
)
```

2. Review and run the `-- load tuesday query` statement: no column list this time, because the table now exists. `LOAD DATA INTO` appends; `LOAD DATA OVERWRITE` would replace.

```
LOAD DATA INTO class.usage_sql
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-21.csv']
)
```

3. Run the `-- verify query` statement:

```
SELECT
  event_date,
  COUNT(*) AS meetings,
  SUM(minutes) AS total_minutes
FROM class.usage_sql
GROUP BY event_date
ORDER BY event_date
```

Expected result. Each of the two dates has **1,000,000** meetings. Open **Job information** for either load: **Bytes billed: 0 B**. `LOAD DATA` is a SQL statement, but it runs the same free load machinery as the console form and `bq load` — which makes it the loading tool for places where everything must be SQL: scheduled queries, stored procedures, and scripts that create, load, and transform in one place.

Gotcha box

- **Run the statements in order, one at a time.** Monday's statement creates the table; Tuesday's depends on it existing. Run, wait, run.
- `usage_sql` **vs** `usage`**.** Activity 1's table is still around; these statements deliberately write a separate table so you can compare the two paths.
- **Free load, billed query.** Don't let "0 B" confuse you later: loads are free, queries bill by bytes scanned. The verify query is the only thing in this activity that could ever cost anything.

If you finish early

- Sketch the scheduled query that would load *today's* vendor file automatically: `LOAD DATA` with a URI built by `FORMAT_DATE`. Which column would you partition `usage_sql` by? Compare your sketch with the scheduled-query demo at the Day-1 close.
- Think through the vendor-change questions: what else do vendors put in numeric columns — empty strings, `null`, `-`, `missing`? What happens on the day the vendor adds a new column to the file?

Self-check

- [ ] `class.usage` shows **2,000,000** rows, and you can find both Load jobs in the job history.
- [ ] You saw Monday's autodetect load fail, and you can explain why (the `N/A` sat outside the sample, so autodetect chose INTEGER and the parse failed at row 90,000).
- [ ] You saw Tuesday's *silent* failure: `minutes` typed as STRING, and SUM failing on it.
- [ ] After the pinned-schema loads: **101,000** rows, exactly **2** NULL minutes, SUM works.
- [ ] `class.usage_history_csv` and `class.usage_history_pq` both show **60,000,000** rows — and you wrote down the two elapsed times.
- [ ] `class.usage_sql` shows **1,000,000** meetings per date, and Job information shows **Bytes billed: 0 B**.

What this replaces in your current stack

- **SSIS flat-file import packages** → `bq load` in a script, or `LOAD DATA` in SQL.
- **The Import and Export wizard** → console Create table from Google Cloud Storage.
- **SQL Agent import jobs** → a scheduled query running `LOAD DATA` (previewed at the Day-1 close).
- **Schema-drift surprises in production** → a pinned schema JSON reviewed in source control, plus `--null_marker` for vendor sentinels.

The rules you just learned

- **Pin your schemas.** Autodetect is a first-look tool; anything recurring gets an explicit schema, versioned like code.
- **Prefer Parquet for bulk.** Columnar, splittable, self-describing — minutes became seconds on the same 60 million rows.
- **Load, don't stream-insert, files.** Batch loads are free and built for files; row-by-row inserts are for live event streams, not vendor drops.
- **Everything is repeatable SQL.** Console clicks and CLI flags are the same job; the SQL form goes into source control, procedures, and schedules.

Asset files for this activity

autodetect_demo.sh

```
# Creates the files for the schema autodetect Do-It-Now:
#   daily_usage_mon.csv     - Monday's big drop (100,000 rows); the vendor
#                             writes "N/A" in the minutes column for meetings
#                             that never started; Monday's N/A is at row 90,000
#   daily_usage_tue.csv     - Tuesday's small drop (1,000 rows); its N/A is
#                             at row 700
#   daily_usage_schema.json - the pinned schema used by the fixed loads
#
# Paste this whole file into a Cloud Shell terminal.

(
  echo "meeting_id,user_id,minutes,event_date"
  for i in $(seq 1 100000); do
    if [ $i -eq 90000 ]; then
```

```
      m="N/A"
    else
      m=$((RANDOM % 120 + 1))
    fi
    printf "m-%06d,u-%03d,%s,2026-07-27\n" $i $((i % 500)) $m
  done
) > daily_usage_mon.csv

(
  echo "meeting_id,user_id,minutes,event_date"
  for i in $(seq 1 1000); do
    if [ $i -eq 700 ]; then
      m="N/A"
    else
      m=$((RANDOM % 120 + 1))
    fi
    printf "m-%06d,u-%03d,%s,2026-07-28\n" $i $((i % 50)) $m
  done
) > daily_usage_tue.csv

cat > daily_usage_schema.json <<'EOF'
[
  {"name": "meeting_id", "type": "STRING"},
  {"name": "user_id", "type": "STRING"},
  {"name": "minutes", "type": "INTEGER"},
  {"name": "event_date", "type": "DATE"}
]
EOF

echo "Created daily_usage_mon.csv, daily_usage_tue.csv, and daily_usage_schema.json"
```

load_data_example.sql

```
-- load monday query
-- same job the console and bq load produce, expressed in SQL; the column
-- list pins the schema the way --schema did on the command line
LOAD DATA INTO class.usage_sql (
  meeting_id STRING,
  user_id STRING,
  minutes INT64,
  event_date DATE
)
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-20.csv']
)

-- load tuesday query
-- the table exists now, so no column list is needed; LOAD DATA INTO
-- appends, LOAD DATA OVERWRITE would replace
LOAD DATA INTO class.usage_sql
FROM FILES (
  format = 'CSV',
  skip_leading_rows = 1,
  uris = ['gs://jwd-gcp-demos/ingest_demo/daily/usage_2026-07-21.csv']
)

-- verify query
SELECT
  event_date,
  COUNT(*) AS meetings,
  SUM(minutes) AS total_minutes
FROM class.usage_sql
```

```
GROUP BY event_date
ORDER BY event_date
```

1.5 Walkthrough: Dataform Repo Tour — a Real Silver→Gold Repository

WALKTHROUGH Day 1 · Mon Oct 5, 2026

Time: **15:10–15:25 (inside the 14:45–15:25 ELT & Dataform block) CT** Length: **15 min**

A guided reading of the public demo repository from the Day 1 walkthrough (3:10–3:25 PM CT): github.com/jwdavis/dataform-demo. Browse it in your browser — no setup needed. This is what a real Silver→Gold repo looks like; your CI/CD instincts (git, PRs, release configs) all apply.

What the repo builds

A small three-layer medallion in BigQuery — the same shape as your target architecture:

Layer	Files	Lands in dataset	Your architecture
Raw (declared)	`definitions/customers_declaration.sqlx`, `orders_declaration.sqlx`, `products_declaration.sqlx`	`dataform_demo_raw_data`	**Bronze** — landed by Datastream/loads (this morning's story)
Staging	`definitions/staging/stg_customers.sqlx`, `stg_orders.sqlx`, `stg_products.sqlx`	`dataform_demo_staging`	**Silver** — cleaned, typed, filtered
Marts	`definitions/marts/daily_sales.sqlx`, `customer_metrics.sqlx`, `product_performance.sqlx`	`dataform_demo_marts`	**Gold** — the marts Power BI points at

Nine nodes total in the compiled graph. Bronze to Gold is six small SQL files.

Read it in this order

1. `dataform.json` — the project settings

Verbatim from the repo:

```
{
    "warehouse": "bigquery",
    "defaultSchema": "dataform_demo",
    "assertionSchema": "dataform_assertions",
    "defaultDatabase": "<project-id>",
    "defaultLocation": "US"
}
```

One JSON file says it all: the warehouse is BigQuery, which GCP project to build in (`defaultDatabase` — `<project-id>` is a placeholder the repo owner replaced), the default dataset, and where assertion results live. Compare that with forty packages each burying their own connection strings.

2. `includes/constants.js` **— names in one place**

A short JavaScript file exporting constants: the project ID and the three dataset names (`DATASET_RAW`, `DATASET_STAGING`, `DATASET_MARTS`). Every sqlx file references these constants instead of repeating literals — the same reason you keep environment configuration out of package XML.

3. `definitions/*_declaration.sqlx` **— Bronze enters the graph**

The raw tables are created and loaded outside Dataform — by Datastream, by scripts, by anything. A declaration tells Dataform "this table exists and is managed elsewhere; let models `ref()` it":

```
// definitions/customers_declaration.sqlx
config {
  type: "declaration",           // tells Dataform this is an externally managed table
  schema: constants.DATASET_RAW,   // the dataset where customers actually lives
  name: "customers"
}
```

This is how your Bronze layer — landed by the migration tooling — becomes a first-class node in the dependency graph instead of an invisible upstream assumption.

4. `definitions/staging/stg_*.sqlx` **— Silver**

Each staging file is `type: "table"` plus cleaning SQL you could write in your sleep — `TRIM`, `LOWER`, `PARSE_DATE`, null filters. The first 14 lines of `stg_customers.sqlx` (a `WHERE` clause filtering null email/name follows in the file):

```
config {
    type: "table",
    schema: constants.DATASET_STAGING,
    description: "Cleaned customer data"
}

SELECT
    customer_id,
    TRIM(customer_name) as customer_name,
    LOWER(TRIM(email)) as email,
    PARSE_DATE('%Y-%m-%d', registration_date) as registration_date,
    UPPER(TRIM(country)) as country,
    CURRENT_TIMESTAMP() as processed_at
FROM ${ref("customers")}
```

The one thing SSIS never gave you: `${ref("customers")}` points at the declared raw table, and from that single function Dataform derives the execution order. No precedence constraints to drag around a designer surface.

5. `definitions/marts/*.sqlx` **— Gold**

The marts are aggregate joins over the staging models: `daily_sales` (orders, revenue, completed/cancelled counts per day), `customer_metrics` (lifetime value, order counts, lifespan per customer), `product_performance`. Each is `type: "table"` in the marts dataset, selecting from `${ref("stg_orders")}` and friends. This is the layer Power BI will query.

What the repo deliberately does not have: assertions

The repo ships no assertion files — data-quality tests are something you add. The shape of one, dropped into a staging model's config block (this is the snippet added live in the walkthrough):

```
assertions: {
  uniqueKey: ["customer_id"],
  rowConditions: ['email IS NOT NULL']
}
```

Assertions run with the pipeline and fail the run — the "how do we know the data's good" question, answered where the data is made. You will add one yourself in the lab if you finish early.

Also in the repo

`misc/` holds the CSV load scripts and a Terraform configuration that provisions the GCP side (Dataform repository, git connection, service account, datasets). Browse it if you are curious — you do not need to run any of it for class. After class, the repo README walks through forking it and provisioning your own copy with Terraform: a good first exercise for whoever prototypes back home.

Why this matters for your migration

- The compiled graph you saw — declarations → staging → marts, edges drawn from `ref()` calls — **is** the Silver→Gold pipeline as code. Reviewable in a pull request, versioned in git.
- Development workspace = feature branch. Release configuration = your deployment. Scheduled invocation = SQL Agent. The same discipline you run for SQL Server deployments today, applied to SQL in git.
- Constants in one file, datasets declared once, execution order derived — nothing to babysit between "merge" and "tables rebuilt."

Self-check

- [ ] I can explain what `dataform.json` and `includes/constants.js` each own.
- [ ] I can explain why the raw tables are *declared* rather than *created* in this repo.
- [ ] I can trace one path from a declaration to a mart (`customers` → `stg_customers` → `customer_metrics`) and name the Bronze/Silver/Gold layer of each node.
- [ ] I could write an assertion block for "order_id unique, quantity > 0" without looking it up.

Next

Build the same bones yourself in the Dataform lab (3:25–4:05 PM CT): repo, workspace, sqlx, execute.

- Docs: Dataform overview · Assertions
- Repo: https://github.com/jwdavis/dataform-demo

1.6 Lab: Create and Execute a SQL Workflow in Dataform

LAB **Day 1 · Mon Oct 5, 2026** Time: **15:25–16:05 CT** Length: **40 min**

The anchor lab of Day 1 (Mon Oct 5, 3:25–4:05 PM CT): you build a working Dataform SQL workflow yourself, right after the demo repo walkthrough. The lab's step-by-step instructions live in the Google Skills classroom — this companion orients you, flags what commonly goes wrong, and connects the lab to your migration. It does not replace the lab instructions.

What you'll do

The lab (**Create and Execute a SQL Workflow in Dataform**, Course 1, Module 4, in the Skills classroom) has four objectives:

1. Create a Dataform repository.
2. Create and initialize a Dataform development workspace.
3. Create and execute a SQL workflow.
4. View execution logs in Dataform.

Why it matters for your migration

Your target architecture names Dataform as the Silver→Gold ELT tool. Everything else today is context; these 40 minutes are the thing you will actually do at work. You have just watched a real raw → staging → marts repository end to end (see the repo tour); now you build the same bones — repo, workspace, sqlx, execute — with your own hands, in your own lab project.

Before you start

- Launch the lab from the Skills classroom; it provisions your GCP project and credentials. Work in that lab-provided project, not any other you may see.
- **Check the project picker** at the top of the console every time you sign in — the console loves to default to the wrong project. If lab resources seem "missing," look here first.
- Dataform lives inside the BigQuery console (BigQuery Studio → Dataform). Console navigation shifts occasionally; if the menu has moved, search "Dataform" in the console search bar.
- If the console prompts you to **enable an API** (BigQuery, Dataform), click Enable — that is normal on a fresh project.
- When the lab asks for a **region**, use the one the lab specifies and use it consistently. A repository in one region and datasets in another surfaces later as empty executions and confusing errors.

What you'll build

The arc, so the clicks make sense:

1. A **staging view** over the source data — Silver-layer cleaning logic.
2. A **mart table** that references it with `ref()` — a Gold-layer aggregate.
3. Dataform **compiles the dependency graph** from your `ref()` calls — watch for it.

4. You **execute the workflow** and read the logs — Dataform runs the actions in dependency order and records each one.

Staging view → mart table → graph → run. The same skeleton as the walkthrough repo, smaller.

While you work — pointers

- **The compiled graph** is a tab inside your development workspace. It lags a beat after you edit a sqlx file — if a new model does not appear, wait a few seconds before assuming the file did not save.
- **Executing is not committing.** Running the workflow builds tables and views in BigQuery. Committing versions the code in git. Neither one is a deployment — at work, deployment is a release configuration running off the repo (the 4:05 PM automation block shows the trigger).
- **Execution logs** show each action's status, duration, and the exact compiled SQL BigQuery ran. When something turns red, that log is the first place to look — same instinct as opening the SQL Agent job history.

For the CI/CD-minded

This lab tends to trigger the sharpest questions of the day from teams that already run deployment pipelines. Short answers, so you can keep building:

- **"Is a workspace a branch?"** Effectively, yes: an isolated development copy of the repo, backed by git — your feature branch. You commit and push changes; releases run off the repository, not off your workspace.
- **"Where is the git remote?"** The lab's repository starts as local git. The walkthrough repo is connected to GitHub — which is exactly what you would do at work.
- **"Can two people edit at once?"** Separate development workspaces, merged through git — the same answer as your current shop.
- **"How does this get to prod?"** Release configurations (a git commit-ish compiled into a result) plus scheduled workflow invocations. Your deployment pipeline, translated — Dataform slots into the discipline you already have; it does not ask you to abandon it.

Common gotchas

- **Workspace looks empty or "broken":** you missed the **Initialize workspace** button. This is the number-one stall in this lab — the workspace shows no files until you initialize it.
- **Region mismatch** at repository creation → executions appear to run but results look empty or land somewhere unexpected. Fix: recreate the repository in the correct region.
- **IAM error on execution:** executions run as a service account; if it lacks BigQuery permissions the run fails with a permissions error. Flag the instructor rather than burning ten minutes on it.
- **"My edit didn't take":** compiled-graph lag. Confirm the file saved, give the graph a few seconds, then reload.
- **Everything looks missing:** the project picker. It is always the project picker.

If you finish early

Expect to — SQL people fly through this lab. In rough order of value:

1. **Add an assertion** to one model — a `uniqueKey` or a non-null `rowConditions` check — and re-run. Watch it appear as a node in the compiled graph and as a result in the execution logs. (A few lines of config; the repo tour shows the exact shape.)
2. **Flip a model to** `type: "incremental"` and reason about when you would use it — think of your largest source table from this morning's sizing discussion.
3. **Interrogate the compiled graph:** find the mart you just made. What depends on it? What does it depend on? That answer is what your code review will look like at work.

Self-check

- [ ] Dataform repository created — in the lab project, in the region the lab specified.
- [ ] Development workspace created **and initialized** (files visible).
- [ ] Staging view and mart table defined in sqlx, wired together with `ref()`.
- [ ] Workflow executed successfully — all actions green.
- [ ] Execution logs opened and read, including the compiled SQL.
- [ ] Compiled graph located and explored.
- [ ] Stretch: assertion added and observed in a run.

What this replaces in your current stack

The hand-sequenced transform packages, the "run these three stored procedures in order" SQL Agent steps, and the morning-after manual validation queries. You keep: SQL, git, code review, scheduled deployments. You lose: the designer surface, the shared scheduler box, and validation that lives outside the pipeline.

Debrief, in one sentence: you just did ELT the way your target architecture will do it — SQL models in git, compiled into a dependency graph, executed in order, tested where the data is made.

1.7 Activity: Schedule a Daily Overwriting Query — DIN §29

ACTIVITY **Day 1 · Mon Oct 5, 2026** Time: **16:20–16:30 CT** Length: **10 min**

Guide to the Day 1 closing demo (4:20–4:30 PM CT): five minutes, one click — a query on a daily schedule that rebuilds a derived table. Watch it in class, then replicate it after class with the SQL file in this folder. Idea from the Do-It-Now (DIN) activities page, §29 "Exploring derived tables," step 5.

What you'll do

See — and later repeat — the simplest automation on Google Cloud: take an aggregation your analysts run a hundred times a day, materialize it once into a derived table, and schedule the query to rebuild that table every day at 12:01 AM with write preference **overwrite**.

Why it matters for your migration

- The derived table is the cost story in miniature: same answer, a fraction of the bytes scanned. That is the entire case for your **Gold-layer data marts** feeding Power BI instead of letting reports hammer the base tables.
- The scheduling click is your **nightly SQL Agent job, translated**: a SQL statement, on a schedule, overwriting a table. No scheduler box, no agent service, no SSIS package.
- It is also the honest bridge to Day 2: hand-rolled derived tables go stale and multiply. Tomorrow, Dataform-managed marts replace them — same idea, governed.

How to schedule a query in the BigQuery console

1. Open **BigQuery Studio**, paste your query into the editor, and **run it once** to confirm it works.
2. Above the editor, click **Schedule** → **Create new scheduled query**.
3. Give it a name and set it to repeat **daily at 12:01 AM** (the DIN §29 step 5 schedule).
4. For the destination, choose your dataset — your `class` dataset from this morning (DIN §3) — and a table name, for example `march_zip_sales`.
5. Set the write preference to **Overwrite table** (`WRITE_TRUNCATE`).
6. Save. The job now appears under **Scheduled queries** in the BigQuery console — open it and use the manual-run option once to prove it fires.

What "overwrite destination" means

Every run replaces the table's entire contents — `TRUNCATE` + `INSERT` as one habit. The table is rebuilt from scratch each night, so re-runs are idempotent: no duplicates, no merge logic. (The alternative, **Append**, adds rows on top — right for accumulating history, wrong for a rebuilt snapshot.)

Why this is the cheapest automation in GCP

A scheduled query is serverless: nothing to provision, patch, or babysit. You write SQL; BigQuery runs it on the clock. It is the floor of the automation ladder — the M5/M6 notes show the rungs above it (Cloud Scheduler, Dataform release configurations, Composer).

Try it after class

The file `nested_queries.sql` (printed in full at the end of this activity) — unchanged from the course demo repository — holds the example queries the DIN §29 activity is built around: March 2018 sales-by-zip queries against the shared nested/repeated demo table.

1. Open the file and take the first query (sales by zip for March, from the `nested_once` table).
2. Replace the `<project-id>` placeholder with the shared demo project — your instructor will confirm it in class (it is the `roi-bq-demos` project you starred this morning).
3. Run it once in your lab project, writing results into your `class` dataset (destination `class.march_zip_sales`).
4. Schedule it using the steps above: daily, 12:01 AM, overwrite.

Two things to notice while you run it:

- `UNNEST(line_items)` — the demo table is nested/repeated: one order row carries an array of line-item structs. Different from your flat SQL Server tables, and one of BigQuery's genuinely new tricks worth a second look.
- **Bytes processed** (the editor shows it before and after a run). The full DIN §29 activity compares querying the large base table against the small derived one — the base demo table is big, so glance at the scan size. That number is your Power BI cost argument. The scheduled overwrite itself is low cost.

Common gotchas

> - **API prompt:** scheduled queries ride on the BigQuery Data Transfer Service. If the console asks to enable it, click Enable and continue.
> - **Destination dataset must exist first.** Your `class` dataset does (you created it this morning); any other dataset must be created before a schedule can target it.
> - **Overwrite vs append:** double-check the write preference. An accidental append doubles the table every night.
> - **Project/location mix-ups:** while learning, keep the source data, the destination dataset, and the schedule in your lab project and its default location — cross-project and cross-location combinations are where first attempts go wrong.
> - **It runs as you by default:** fine in a lab project. At work you would run it as a service account — your SQL Agent proxy instinct is the right one; keep it.

Go further (the rest of DIN §29)

The full activity (steps 2–4, about 20 minutes) has you build the derived table, then time the same aggregation against the full table and against the derived table, comparing duration and bytes processed. That arms you with real numbers for the "why Gold marts" conversation back home. The DIN page also

asks the right skeptical questions — derived tables go stale and duplicate data — which is exactly why Day 2 replaces hand-rolled derived tables with Dataform-managed marts, and where partitioning, clustering, and materialized views enter the conversation.

Self-check

- ☐ I can find **Schedule** → **Create new scheduled query** in BigQuery Studio.
- ☐ I can explain what overwrite (`WRITE_TRUNCATE`) does, and when I would choose append instead.
- ☐ I have a job visible under **Scheduled queries** and have seen it run once.
- ☐ I can name the SQL Agent equivalent of every part of this setup (job step = the query, schedule = daily 12:01 AM, destination = the derived table).
- ☐ I can explain to a Power BI teammate why querying the derived table is cheaper than hitting the base table.

What this replaces in your current stack

The nightly SQL Agent job that runs T-SQL to rebuild a reporting table — minus the job engine, the proxy account wrangling, and the server it ran on. Keep the instinct ("materialize what analysts hammer, refresh it on a schedule"); the platform absorbed the plumbing.

- The Do-It-Now (DIN) activities page: https://roitraining.github.io/gcp-demos/bigquery.html (§29, step 5; your `class` dataset is from §3)
- Docs: Scheduling queries in BigQuery
- Example SQL: `nested_queries.sql`

Asset files for this activity

nested_queries.sql

```
-- standardSQL
-- find sales/zip for march from nested_once table
WITH
  orders AS (
  SELECT
    cust_zip,
    prod_price * qty AS line_item_subtotal
  FROM
    `<project-id>.bq_demo.nested_once`,
    unnest(line_items)
  WHERE
    order_date >= "2018-03-01"
    AND order_date <= "2018-03-31")
SELECT
  cust_zip,
  SUM(line_item_subtotal) as zip_sales
FROM
  orders
GROUP BY
  cust_zip
order by
  zip_sales desc
```

```
-- standardSQL
-- find sales/zip for march from nested/partitioned
WITH
  orders AS (
  SELECT
    cust_zip,
    prod_price * qty AS line_item_subtotal
  FROM
    `<project-id>.bq_demo.table_nested_partitioned`,
    unnest(line_items)
  WHERE
    order_date >= "2018-03-01"
    AND order_date <= "2018-03-31")
SELECT
  cust_zip,
  SUM(line_item_subtotal) as zip_sales
FROM
  orders
GROUP BY
  cust_zip
order by
  zip_sales desc

-- standardSQL
-- find sales for 6 months in 8754 from nested
WITH
  orders AS (
  SELECT
    cust_zip,
    prod_price * qty AS line_item_subtotal
  FROM
    `<project-id>.bq_demo.nested_once`,
    UNNEST(line_items)
  WHERE
    order_date >= "2018-01-01"
    AND order_date <= "2018-06-30"
    AND cust_zip=8754)
  SELECT
    cust_zip,
    SUM(line_item_subtotal) AS zip_sales
  FROM
    orders
  GROUP BY
    cust_zip
  ORDER BY
    zip_sales DESC

-- standardSQL
-- find for 6 months in 8754 from nested/partitioned
WITH
  orders AS (
  SELECT
    cust_zip,
    prod_price * qty AS line_item_subtotal
  FROM
    `<project-id>.bq_demo.table_nested_partitioned`,
    UNNEST(line_items)
  WHERE
    order_date >= "2018-01-01"
    AND order_date <= "2018-06-30"
    AND cust_zip=8754)
  SELECT
    cust_zip,
    SUM(line_item_subtotal) AS zip_sales
```

```
  FROM
    orders
  GROUP BY
    cust_zip
  ORDER BY
    zip_sales DESC

-- standardSQL
-- find for 6 months in 8754 from nested/partitioned/clustered
WITH
  orders AS (
  SELECT
    cust_zip,
    prod_price * qty AS line_item_subtotal
  FROM
    `<project-id>.bq_demo.table_nested_partitioned_clustered`,
    UNNEST(line_items)
  WHERE
    order_date >= "2018-01-01"
    AND order_date <= "2018-06-30"
    AND cust_zip=8754)
  SELECT
    cust_zip,
    SUM(line_item_subtotal) AS zip_sales
  FROM
    orders
  GROUP BY
    cust_zip
  ORDER BY
    zip_sales DESC
```

DAY 2 · TUESDAY, OCTOBER 6, 2026

Day 2 — Lakehouse & Medallion

Yesterday's mechanics become your architecture: the lakehouse, Bronze/Silver/Gold, the Power BI last mile, and a capstone you take back to work.

Day 2 Schedule

"Your target operating model: lakehouse & medallion" · 9:00 AM–4:30 PM CT · virtual. Day 2 turns yesterday's mechanics into your architecture: the lakehouse, Bronze/Silver/Gold on Google's own slides, the Power BI last mile, and a capstone you take back to work.

Time	Block	What you'll walk away with
09:00–09:15	**Recap quiz**	A fast, low-stakes game of "which GCP tool replaces which piece of your stack?" — extracts, Agent jobs, SSIS and all
09:15–09:45	**Lake, warehouse, lakehouse**	The vocabulary of your own architecture doc, pinned down: what a lakehouse is and why it's your target shape
09:45–10:30	**Building the lakehouse**	Why Cloud Storage + open formats (Parquet, Iceberg) is your Bronze foundation, and how federated queries reach data you haven't moved yet
10:30–10:45	**Break**	
10:45–11:15	**Lab: federated queries** — *Federated Query with BigQuery*	A live join between BigQuery data and an operational database — your linked-server/`OPENQUERY` instinct, working on GCP
11:15–11:35	**Hands-on: external tables — Bronze made tangible**	A queryable table over Hive-partitioned Parquet in the `jwd-gcp-demos` bucket — no load job, partition pruning on raw files (DIN §30)
11:35–12:00	**Modernizing your warehouse**	Your SQL Server pain points, answered: partitioning, clustering, and how BigQuery compute (slots) changes the tuning game
12:00–13:00	**Lunch**	
13:00–13:45	**Lab: external data & Iceberg** — *Querying External Data and Iceberg Tables*	A BigQuery Lakehouse (Iceberg) table over files in Cloud Storage, queried without loading anything — the Bronze pattern, hands-on
13:45–14:00	**Iceberg deep-dive**	What's under an Iceberg table: snapshots, partitioning, and what BigQuery actually does when you hit RUN
14:00–14:35	**Governance + the medallion (Bronze/Silver/Gold)**	Google's own Bronze/Silver/Gold zone slides held up next to your design — plus Knowledge Catalog, sensitive-data protection, and fine-grained access: the pieces that make one source of truth safe
14:35–14:50	**Break**	
14:50–15:20	**ML in BigQuery + the Power BI last mile**	A quick look at BigQuery ML (so you know it exists), then the part you asked for: your Gold marts feeding the Power BI you already have — Import vs. DirectQuery, auth, gateways
15:20–15:50	**Capstone: map one of your real jobs**	A filled-in migration map for one real DOM DoIT extract job — backlog draft zero, on paper, yours to keep

Time	Block	What you'll walk away with
15:50–16:10	**Course summary**	The two days retraced against your migration map, one more time, fast
16:10–16:30	**Q&A + what's next**	Open questions (the parking lot gets emptied), and where the follow-on *Data Warehousing with BigQuery* course goes deeper

Worth knowing before you arrive

- **Every hands-on block above links to its section in this workbook.** Read the section before the block starts — it carries the gotchas.
- **Bring a real job to the capstone.** Think ahead about one extract job your team owns — not the biggest, not the simplest, the most typical. At 15:20 your group maps it onto the GCP target architecture using the migration map.
- **Labs live in the Google Skills classroom**, same as yesterday — temporary projects, timers, disposable sandboxes.
- **"DIN" = the Do-It-Now activities page:** https://roitraining.github.io/gcp-demos/bigquery.html — today uses §30 (external tables over Hive-partitioned Parquet).
- **The Power BI segment is built for you.** The standard course never mentions Power BI; this custom segment exists because your endgame is Gold marts feeding the reports the state already runs on. Details live in the Power BI ↔ BigQuery connector guide.

Reference cards to keep open today

- Migration map — the one-page architecture card; have it open during the medallion block and the capstone
- T-SQL → BigQuery SQL — the Iceberg lab is where dialect differences bite
- Glossary — lakehouse vocabulary lands fast today; look things up as they fly by

2.1 Activity: Recap Quiz — Which GCP Tool Replaces Which Piece of Your Stack?

ACTIVITY **Day 2 · Tue Oct 6, 2026** Time: **09:00–09:15 CT** Length: **15 min**

Day 2 (Tue Oct 6), 09:00–09:15. A no-stakes group warm-up: before the lakehouse modules start, prove that Day 1 stuck — in your own vocabulary.

How it works

Your instructor reads each question aloud. Confer with your table for about 30 seconds, then someone answers for the group. Nothing is scored and there are no trick questions — the point is retrieval practice: naming, out loud, what each piece of your current SQL Server stack becomes on Google Cloud. Jot your answers down as you go; the key is at the bottom (no peeking).

The questions

1. **Your nightly extract jobs** — the SQL Server Agent tasks that dump tables to files or stage them to another server. Which GCP service replaces them for ongoing database replication?
2. **SQL Server Agent** — the scheduler that fires your jobs at 2 AM. What's the GCP equivalent?
3. **SSIS** — your extract-transform-load packages. What replaces it on GCP?
4. **Your staging file drops** — CSVs arriving from counties and vendors. Where do they land first in the target design, and what loads them?
5. **Your linked servers and cross-database queries.** What's the BigQuery way to query data that lives somewhere else?
6. **Your T-SQL stored procedures and deployment scripts** that build reporting tables. Where does that discipline live in the target?
7. **Your SSAS cubes / pre-aggregated reporting tables.** What's the BigQuery analogue?
8. **Yesterday's race:** the 60-million-row file that took ~2 minutes to load as CSV and seconds as Parquet. What was the lesson?
9. **Who pays when a query runs,** and what does a query actually bill on?
10. **Bronze, Silver, Gold** — in one sentence each, what are they in *your* target design?

Answers

1. **Datastream** — change data capture (CDC) that streams database changes into BigQuery continuously: log-based, no triggers, no production load. That's why it beats your nightly window — yesterday, a row changed in the source while we ate lunch, with no extract job at all. (BigQuery Data Transfer Service is the right answer only for *file/SaaS* sources, not database CDC.)
2. **Cloud Scheduler** — plus **Dataform release configurations** on the transform side. For a one-statement job, a BigQuery **scheduled query** is the cheapest version — that was the closing demo yesterday: one click, a daily overwriting query, no pipeline.

3. Two answers, and the distinction matters: **Dataform** for ELT — transform in SQL, inside BigQuery, your Silver→Gold path — and **Dataflow** for when SQL genuinely isn't enough (streaming, heavy custom logic). Your architecture doc names Dataform; Dataflow is the escape hatch, not the plan.
4. They land in **Cloud Storage** — the Bronze layer — and load via **Storage Transfer Service,** `bq load`**,** `LOAD DATA` **SQL**, or scheduled through **BigQuery Data Transfer Service**. GCS first because it's a cheap, durable, replayable raw copy: the lake holds what the warehouse hasn't modeled yet.
5. **Federated queries over external connections** (this morning's lab) and **external tables** over GCS (the 11:15 activity). The trade-off versus loading: no copy and always current, but no BigQuery storage optimizations — and the source system takes the load.
6. **Dataform** — SQLX definitions, a dependency graph, assertions, release configurations, all in git, wired to your CI/CD. A Dataform assertion replaces the hand-rolled validation queries and job-step checks you run after loads today.
7. **Materialized views** — BigQuery maintains the refresh for you, with nobody running a cube-processing job. More broadly: your **Gold-layer marts**, built and scheduled by Dataform.
8. **Open columnar formats matter.** Parquet on GCS is the Bronze foundation — and it's what Iceberg and BigLake build on (today's story). If your current extracts land as CSV, changing the format is a free, immediate win in the migration.
9. Queries bill on **bytes scanned** under on-demand pricing; **loads are free** — every load yesterday showed "Bytes billed: 0 B." Which makes the cheapest automation you built yesterday the daily overwriting scheduled query: one statement, no pipeline. (Slots and flat-rate exist; the data-warehousing course goes deeper.)
10. **Bronze** = raw lands as-is (GCS files, Datastream replicas, external/BigLake tables). **Silver** = cleaned, conformed, deduplicated BigQuery tables. **Gold** = business-ready marts that feed Power BI. Dataform does Silver→Gold — and your Power BI reports touch Gold only. Today we make all three concrete.

2.2 Lab: Federated Query with BigQuery and AlloyDB

LAB **Day 2 · Tue Oct 6, 2026** Time: **10:45–11:15 CT** Length: **30 min**

Day 2 (Tue Oct 6), 10:45–11:15. The lab's own step-by-step lives in your Google Skills classroom seat (listed there as "Federated Query with BigQuery"); this companion is the orientation around it — what you're building, why, and what tends to go wrong.

The one thing to remember

> **A federated query is BigQuery's version of a linked server with `OPENQUERY`.**
>
> - The **connection resource** = the **linked server definition**: server, credentials, one-time setup.
> - `EXTERNAL_QUERY(connection_id, '...')` = `OPENQUERY`: the string inside the quotes is shipped to the remote system and executed *there*, in *its* SQL dialect.
> - And just like `OPENQUERY`: there is no browsing the remote tables — you query blind.

What you'll do

- Connect BigQuery to an AlloyDB for PostgreSQL database (AlloyDB plays the role of a live operational store — it is not your migration target).
- Run your first `EXTERNAL_QUERY` against it.
- Join historical analytical data (in BigQuery) with live transactional rows (in AlloyDB) in a single SQL statement.

Why it matters for your migration

During your migration, not everything moves at once. Federation means the SQL Server systems that are still operational stay reachable from BigQuery **without copying them first** — no extract job built just to join operational tables against warehouse data. This is the pattern that lets you migrate in phases instead of betting on a big-bang cutover.

The honest trade-offs, so you apply it deliberately: the source system takes the query load, results don't get BigQuery storage optimizations, federated queries are read-only, and on-demand billing charges for the bytes returned from the external side. Federation is a bridge, not a destination — Silver/Gold still belong in managed tables.

Before you start

- Launch the lab from your seat in the Skills classroom and sign in to the console with the **lab-provided credentials**, not your own account.
- Check the **project picker** at the top of the console: the lab spins up its own temporary GCP project, and everything you build must go there.
- Note the region the lab instructions give you, and keep every resource you create in that region.

While you work — pointers, not steps

- **The connection setup is the "linked server definition" moment.** New concept, one-time work: region, connection ID, credentials. Go slowly here; once it exists, everything else is just SQL.
- **API-enable and consent prompts are expected.** If the console asks you to enable an API (for example the BigQuery Connection API) or confirm a consent screen, click through — that's normal in a fresh lab project.
- **Your first `EXTERNAL_QUERY`:** the inner statement is **PostgreSQL**, executed on the AlloyDB side. BigQuery syntax inside the quotes fails — this is the single most common trip point. Watch your quoting too: single quotes inside the inner query must be escaped (double them, or wrap the whole inner query in triple single quotes `'''...'''` the way Google's own examples do).
- **The payoff is the join:** historical/analytical data on the BigQuery side ⋈ live transactional rows on the AlloyDB side, one statement. If you get nothing else working, get this — a working connection you can re-create later is the real takeaway.

Common gotchas

- **Wrong project in the picker** — you created the connection in the wrong project and now nothing in the lab matches. Check the picker first whenever something "isn't there."
- **Connection ID typos in `EXTERNAL_QUERY`** — it must match exactly, in the form `region.connection-id` (for example `'us.my-connection'`).
- **Region alignment** — the connection resource's location must match where the query runs, and a single-region BigQuery setup can only federate to sources in the same region. If the lab says one region, use it everywhere.
- **BigQuery SQL inside the quotes** — the inner string is PostgreSQL. Backticks, GoogleSQL functions, and T-SQL habits all fail in there.
- **"Why can't I see the AlloyDB tables in the Explorer pane?"** — because federation is query-only. There is no table browsing; you write the inner query blind, exactly like `OPENQUERY`.

If you finish early

- Run the inner PostgreSQL query on its own (just the string inside the quotes) to prove to yourself which side executes it.
- Add a filter inside the inner query — for example a `WHERE` or a `LIMIT` — and watch the remote side do the work before BigQuery sees a row.
- List the remote database's metadata through the same connection:

  ```
  SELECT * FROM EXTERNAL_QUERY("region.connection-id",
    "select * from information_schema.tables;");
  ```

  (swap in your connection ID — this is the PostgreSQL `information_schema`, not BigQuery's).
- Open the job's **Execution details** after the join and find what BigQuery reports for the federated step.

Self-check

- [ ] I can see my connection resource under **External connections** in the BigQuery Explorer pane.
- [ ] My first `EXTERNAL_QUERY` returned rows from AlloyDB.
- [ ] I joined BigQuery historical data with live AlloyDB transactional rows in one query.
- [ ] I can explain which part of the query runs where: the inner string is PostgreSQL on AlloyDB; everything outside `EXTERNAL_QUERY` is GoogleSQL in BigQuery.
- [ ] I can name what this maps to in my current stack: a linked server definition plus `OPENQUERY`.

What this replaces in your current stack

- **Linked servers, four-part names, and `OPENQUERY`** — the connection resource plus `EXTERNAL_QUERY` covers the same ground.
- **The nightly "extract the operational tables just so we can join them" jobs** — during the migration you query data where it lives and load it only when there's a reason to.

The debrief in one line: querying data where it lives — that is the lakehouse promise. Your Bronze lands in GCS/BigQuery, but anything still operational on the SQL Server side stays reachable during the migration. You don't have to move it all first.

Dig deeper

- Introduction to federated queries — region rules, quotas, pricing, SQL pushdowns
- AlloyDB federated queries — including the `information_schema` trick above
- AlloyDB overview — what the lab's stand-in operational store actually is
- Next: the same trick against files in Cloud Storage — Exploring external tables

2.3 Activity: External Tables — Bronze Made Tangible (DIN §30)

ACTIVITY **Day 2 · Tue Oct 6, 2026** Time: **11:15–11:35 CT** Length: **20 min**

Day 2 (Tue Oct 6), 11:15–11:35, instructor-led follow-along. You will point BigQuery at a bucket of Hive-partitioned Parquet files and query them as a table — with no load job, no SSIS package, and no staging table. Adapted from §30 of the Do-It-Now (DIN) activities page.

What you'll do

1. Inspect a public GCS bucket (`gs://jwd-gcp-demos/orders_partitioned/`) laid out in Hive partition style.
2. Create a BigQuery **external table** over it, named `ext_part`, in your `class` dataset.
3. Run three small queries and watch **bytes processed**: a full scan, a filter on a data column (still a full scan), and a filter on the partition column (bytes drop dramatically).

Cost: effectively free — the DIN page badges this activity low-cost, and the scanned bytes on this dataset are tiny (three small queries; pennies at on-demand rates).

Why it matters for your migration

This is Bronze made tangible. Open-format files sit in a bucket, partitioned by date, and are immediately queryable in standard SQL — zero loading. Your architecture doc's Bronze layer is exactly this.

- Today, querying a county file drop means an SSIS package and a staging table first. Here **the file *is* the table.**
- For the M&O team: there is no database to patch and no service to keep alive. Operating Bronze is bucket IAM and lifecycle policies, not servers.
- The honest trade-off: external tables get **no BigQuery storage optimizations** — no native clustering, slower repeated scans, and the source of truth lives outside BigQuery. That is precisely why your Silver and Gold become *managed* tables built by Dataform. Bronze is for landing and exploring; Silver/Gold are for serving.

Before you start

- You need the `class` **dataset** you created on Day 1 morning (DIN §3). If you don't have it: in the BigQuery **Explorer** pane, click your project, then the three-dot menu → **Create dataset** → Dataset ID `class` → **Create dataset**.
- Open the **BigQuery console**.
- Open the SQL file for this activity — printed in full at the end of this activity, `external_hive_example.sql` (byte-identical to the course repo's copy; the DIN page links the same file).

- In another browser tab, open the bucket browser: https://console.cloud.google.com/storage/browser/jwd-gcp-demos/orders_partitioned

Step 1 — Look at the bucket first

Browse `gs://jwd-gcp-demos/orders_partitioned/` before creating anything, and answer for yourself:

- How many "directories" are there in the bucket?
- How many files does each directory hold?
- What does the naming of the directories imply?

What you are seeing is a **Hive-partitioned dataset living in a data lake on GCS**: directories named `order_date=YYYY-MM-DD/` — the Hive `key=value` partition layout — each holding Parquet file(s). Spotting that naming convention yourself is the payoff: the *path* is metadata.

Step 2 — Create the external table

1. In the BigQuery **Explorer** pane, select your `class` dataset.
2. From its three-dot menu, select **Create table**.
3. In **Create table from**, select **Google Cloud Storage**.
4. In **Select file from GCS bucket or use a URI pattern**, enter `jwd-gcp-demos/orders_partitioned/*`
5. In **File format**, select **Parquet**.
6. Check **Source data partitioning**. (Leave the partition inference mode at its default, automatically infer types.)
7. In **Select Source URI Prefix**, enter `gs://jwd-gcp-demos/orders_partitioned/`
8. Under Destination: in **Table**, enter `ext_part`; set **Table type** to **External table**.
9. In **Schema**, select **Auto detect**.
10. Click **CREATE TABLE**.
11. Open the new table's details. Notice what is *not* there: no storage size of its own. The table is a metadata pointer at `gs://jwd-gcp-demos/orders_partitioned/` — delete the table later and every file in the bucket is untouched.

Step 3 — Query it, and watch the bytes

Run the three queries from `external_hive_example.sql` one at a time. After each run, open the job's **Execution details** and note how much data was processed.

1. **Query the external table** — `SELECT * FROM class.ext_part` Full scan: BigQuery opens every Parquet file in every partition folder. Rows returned = the whole dataset.
2. **Query with a WHERE on a data column** — `WHERE order_num="68610383-54"` Still scans (almost) everything. `order_num` is a *data* column, not the partition key, so BigQuery must open every file to find that order. A `WHERE` clause does not automatically save you money — say that sentence to your neighbor.

3. **Query on the partition column** — `WHERE order_date="2018-01-01"` Bytes processed drops dramatically. Because you checked **Source data partitioning**, BigQuery inferred `order_date` from the folder names and reads **only the matching partition's files**.

That third query is the lesson: **partition pruning on raw files in a bucket — and no load job ever ran.**

Common gotchas

- **The two URI fields want different formats.** The file pattern is `jwd-gcp-demos/orders_partitioned/*` — with the `*` wildcard, no `gs://`. The Source URI Prefix is `gs://jwd-gcp-demos/orders_partitioned/` — with `gs://`, no wildcard. Swap or blur them and table creation fails.
- **Don't forget the `*`.** Without the wildcard, BigQuery can't match the files inside the partition folders.
- **Don't skip the "Source data partitioning" checkbox.** Without it, `order_date` never becomes a column — the third query errors on an unrecognized name instead of pruning partitions.
- **Schema: use Auto detect, and relax.** Parquet files carry their schema inside the file, so autodetect is safe here — the CSV autodetect surprises you saw on Day 1 don't apply to Parquet.
- **Table type must be External table.** Otherwise BigQuery tries to *load* the data into managed storage instead of pointing at it — the opposite of what this activity is about.
- **The SQL assumes your dataset is literally named `class`.** If yours is named differently, adjust `class.ext_part` in the queries.

Self-check

☐ `ext_part` exists in my `class` dataset, and its details show an external source URI — not stored bytes.
☐ Query 1 returned the full dataset (all partitions, every file).
☐ Query 2 (`order_num`) scanned roughly the same bytes as query 1 — and I can explain why.
☐ Query 3 (`order_date`) processed dramatically fewer bytes — and I can explain why.
☐ I can say why this is "Bronze," and what would turn this data into Silver (a managed, cleaned BigQuery table — built by Dataform).

If you finish early

- Pick another `order_date` folder you saw in the bucket browser and run the partition query for that date; compare bytes processed.
- Run `SELECT order_date, COUNT(*) FROM class.ext_part GROUP BY order_date` and predict the bytes before you run it — why is it a full scan?
- Compare the bytes processed of query 2 and query 3 side by side in your job history. That ratio is the argument you'll make in your own design reviews for partitioning Bronze by date.
- Reopen the table details and study the external data configuration — this is the definition you'd script later with `bq mkdef` or DDL.

What this replaces in your current stack

- **The SSIS package + staging table** you build today before anyone can query a county or vendor file drop. Bronze is queryable the moment files land.
- **The "copy it into the warehouse before it's useful" reflex.** Copy when you have a reason (performance, governance, conformance) — that's Silver/Gold — not as a reflex.
- A preview of this afternoon: Iceberg upgrades exactly this pattern — what if the files in the bucket could also get ACID semantics and schema evolution? Keep `ext_part` in mind.

Dig deeper

- Create Cloud Storage external tables — the console fields you just used, plus the DDL/`bq` equivalents
- Manage external Hive partitioned data — partition layouts, inference modes, pruning rules
- Introduction to BigLake tables — the governed upgrade path for external tables
- The Do-It-Now (DIN) activities page — §30 is this activity; §3 created your `class` dataset

Asset files for this activity

external_hive_example.sql

```
-- query the external table
SELECT
  *
FROM
  class.ext_part

-- query external table with where clause
SELECT
  *
FROM
  class.ext_part
WHERE
  order_num="68610383-54"

--query external table on partition
SELECT
  *
FROM
  class.ext_part
WHERE
  order_date="2018-01-01"
```

2.4 Lab: Querying External Data and Iceberg Tables

LAB **Day 2 · Tue Oct 6, 2026** Time: **13:00–13:45 CT** Length: **45 min**

Companion for the Day 2 (Tue Oct 6) after-lunch lab, 13:00–13:45 (45 min), run from your Skills classroom seat. This is the Bronze-layer lab: you query data in Cloud Storage that is never loaded into BigQuery. The step-by-step instructions live in the lab itself; this file is your orientation, dialect decoder, and gotcha list.

- Classroom: https://www.skills.google/ilt/classrooms/37990
- Related notes: M9 student notes · Iceberg explained · M10 governance + medallion notes

What you'll do

From the lab's own introduction:

- Explore how BigQuery's Lakehouse tables can query data in open-source formats directly within Cloud Storage.
- Create a Lakehouse table that points to an existing Apache Iceberg table.
- Analyze data without moving or duplicating it.

The lab has two stretches: first the DDL that points BigQuery at the existing Iceberg data in a Cloud Storage bucket, then ordinary SQL — filters, aggregations — against the table you just created.

Why it matters for your migration

This is your Bronze layer, hands-on. The pattern from your architecture doc: raw data stays in open formats in object storage, and BigQuery queries it in place. Bronze is not a landing zone you copy out of — it is queryable from minute one. In this morning's DIN activity (§30, external tables) you saw files in a bucket become a read-only table with zero loading; this lab takes the same idea and adds Apache Iceberg's table semantics (metadata, ACID behavior) on top of the files. When you design Bronze for the real project, this lab is the reference experience.

Before you start

- **Launch the lab from the Skills classroom** and wait for your seat to provision. You work in your own **lab-provided GCP project** with lab credentials — not any project you may have used before.
- **Check the project picker** at the top of the console before every create. The console sometimes shows a different project from an earlier session; creating the table in the wrong project is the most common self-inflicted error.
- **Have your `class` dataset ready** — you created it on Day 1 morning (DIN §3). If it's missing, create it now (Explorer pane → three-dot menu next to your project → Create dataset). The table you create needs a dataset to live in.
- If the console asks you to **enable an API** or accept terms along the way, click enable/accept. That is expected in a fresh lab project, not an error.
- Type the DDL yourself at least once rather than pasting it — the muscle memory is the point.

T-SQL → BigQuery SQL: the dialect gotcha box

This is where you'll actually spend your debugging time. BigQuery speaks **GoogleSQL** (standard SQL) — close to T-SQL, different in the places your fingers remember. Keep this box open next to the editor.

Your T-SQL reflex	The GoogleSQL way
`USE mydb;` then bare table names	No `USE`. The dataset is part of the name: `` `project.dataset.table` ``
`[dbo].[Orders]` square brackets	Backticks: `` `class.orders` `` — square brackets are a syntax error
`SELECT TOP 100 ...`	`SELECT ... LIMIT 100` (at the end of the query)
`ISNULL(col, 0)`	`IFNULL(col, 0)` or `COALESCE(col, 0)`
`GETDATE()` / `GETUTCDATE()`	`CURRENT_DATE()` / `CURRENT_TIMESTAMP()`
`DATEADD(day, -7, col)`	`DATE_SUB(col, INTERVAL 7 DAY)`
`DATEDIFF(day, start, end)`	`DATE_DIFF(end, start, DAY)` — argument order flips
`YEAR(col)` / `DATEPART(m, col)`	`EXTRACT(YEAR FROM col)` / `DATE_TRUNC(col, MONTH)`
String concatenation with `+`	`CONCAT(a, b)` or the `
`LEN(col)`, `CHARINDEX(...)`	`LENGTH(col)`, `STRPOS(...)`
`VARCHAR(MAX)`, `NVARCHAR`, `DATETIME2`	`STRING`, `TIMESTAMP` (or `DATETIME` — no timezone)

Two lab-specific notes:

- **The DDL is metadata-only.** Creating a table over external files registers a pointer; no data moves. It should return in a second or two. If your `CREATE` hangs or errors, the statement is wrong (typo in the URI, wrong location, missing connection) — the data is not "slow to load," because nothing loads.
- **No autodetect roulette here.** Yesterday's CSV autodetect pitfall (the `N/A` marker silently flipping a column to STRING) was a CSV problem. Parquet and Iceberg files carry their own schema, so types come out right by construction.

Full function reference when you need it: GoogleSQL functions and operators.

Common gotchas

- **Sixteen people, one lab start time.** Seats and first queries can be slow right at 13:00 while everyone provisions and runs at once. Expect some waiting; don't refresh-spam or re-run in a loop.
- **Table-name collisions.** Everyone follows the same instructions against the same source. If you create a table with the exact name in the lab doc and a classmate's DDL shares your dataset, you'll collide. Suffix your table names with your initials or seat number (e.g., `iceberg_orders_ia`).
- **Copy-pasted URIs lose characters.** A `gs://` URI that line-wraps in the lab doc can drop characters when pasted. The classic symptom: the table creates fine, then queries return 0 rows or "not found." If that happens, re-check the URI character by character against the lab doc.

- **Region mismatch.** The dataset's location and the bucket's location must be compatible. This produces the classic first-lab error message; the fix is to create your dataset in the location the lab specifies — don't freestyle it.
- **Query through the table, not the bucket.** If you find yourself browsing the GCS file listing to answer a question, come back to the SQL console — querying the files as a table is the entire exercise.
- **External table vs. Lakehouse (Iceberg) table, in one sentence:** this morning's external table was a read-only pointer you manage nothing about; a Lakehouse/Iceberg table is still files in GCS, but with table metadata and semantics on top — the path toward governance and ACID behavior. For your design: Bronze/Silver = Iceberg-family tables, not bare external tables.

If you finish early

- Run the same aggregate with and without a filter on the partition column; open **Job information / Execution details** and compare **bytes processed**. Then filter on a non-partition column and compare again. You are watching Iceberg's metadata pruning and predicate pushdown from the M9 slides.
- Deliberately write one query the T-SQL way (brackets, `TOP`, `ISNULL`), watch it fail, and fix it using the dialect box. Breaking it on purpose is the fastest way to learn the deltas.
- Browse the source bucket in the Cloud Storage console and find the Iceberg metadata (manifest) files next to the data files — that metadata is what the next 15-minute block is about.

Self-check

- [ ] Lab seat provisioned, and I confirmed I'm in the lab-provided project (project picker)
- [ ] Lakehouse table created over the existing Iceberg data, named with my suffix
- [ ] First `SELECT` returned rows — with no load job ever running
- [ ] I ran a filtered and an aggregated query and noted bytes processed
- [ ] I can say in one sentence how this differs from this morning's external table (Iceberg metadata adds table semantics over the same kind of files)

What this replaces in your current stack

The SSIS package + staging table you build today before anyone can ask a question of a file drop. Here, the file landing in the bucket *is* the queryable table. It also previews the end of linked-server/`OPENQUERY` patterns: you don't copy data next to the compute to query it.

Debrief: tie it to your Bronze design

Hold onto what just happened: data that was never loaded into BigQuery answered your SQL. Your Bronze zone is exactly this — raw landings in open formats on Cloud Storage, immediately queryable, immutable. The open questions — who can write these tables, who keeps the metadata, how this gets ACID guarantees and governance — are the next two blocks: Iceberg explained (13:45) and then the medallion architecture in M10 (14:00), where the table you just made shows up as the Bronze zone in Google's own zone diagram.

2.5 Deep-dive: Apache Iceberg, Explained

DEEP-DIVE **Day 2 · Tue Oct 6, 2026** Time: **13:45–14:00 CT** Length: **15 min**

Explainer for the Day 2 (Tue Oct 6) 13:45–14:00 block, right after the Iceberg lab. Fifteen minutes on what Iceberg actually is, the three ways BigQuery touches it, and why it's the foundation of your Bronze layer.

What Apache Iceberg is

Apache Iceberg is an **open table format**: a metadata layer that sits over ordinary Parquet (or ORC) files in object storage, like your Cloud Storage buckets. The data files are plain files you can list in the bucket browser. Iceberg adds manifests and snapshots on top of them so that any engine can answer the question "which files make up this table, right now?" Crucially, nobody owns the format — it's an Apache open-source project, and the same Iceberg table can be read by Spark, Trino, Flink, Presto, Hive, and BigQuery at the same time.

That metadata layer buys you the table behaviors you're used to from SQL Server, but over files in a lake: **ACID transactions** (concurrent readers and writers can't corrupt each other; `UPDATE` / `DELETE` / `MERGE` work), **schema evolution** (add, drop, and rename columns without rewriting the table — no "zombie" data), **time travel and rollback** (query the table as of an earlier snapshot, or reset it to a good state), and **hidden partitioning** (partition values are tracked in metadata rather than encoded in folder names, so pruning works without you remembering to filter on the folder convention).

Why does this exist? Plain files in a data lake have no transaction log — two engines writing at once is how tables get corrupted — and Hive-style partitioning forces you to bake partition values into folder paths and remember to filter them by hand (you saw that convention, `order_date=YYYY-MM-DD/`, in this morning's §30 external-tables activity). Iceberg moves both concerns into the metadata. That is what makes a **lakehouse** possible: warehouse-grade tables on lake-grade storage.

The three ways BigQuery touches Iceberg

One line to keep: **"Iceberg" names the file format, not the product. The question that matters is: who can write, and who keeps the metadata?** All three flavors below store Parquet in *your* bucket; they differ in who owns the table metadata and who may write.

Flavor	Who keeps the metadata	Can BigQuery write?	Other engines
Lakehouse table over an existing Iceberg table (what you built in the lab; the BigLake external-table family)	Whoever wrote the Iceberg table (e.g., a Spark job)	No — BigQuery reads in place	The engines that own the table read and write it

Flavor	Who keeps the metadata	Can BigQuery write?	Other engines
BigQuery-managed Iceberg tables ("Iceberg managed tables"; formerly called BigLake tables for Apache Iceberg)	BigQuery's catalog	Yes — full DML, time travel, schema evolution; BigQuery writes Parquet into *your* bucket	Read access via exported Iceberg metadata snapshots / the Storage API
Open-engine access (metadata export / Iceberg REST catalog)	An open catalog	Shares rather than owns	Spark, Trino, and other Iceberg-speaking engines query the same files directly

> **Verify current state — this area evolves quickly.** Product names, preview status, and exactly which engine can write what are among the fastest-moving details in Google Cloud (the managed-table name above has already changed once). Treat this table as the shape of the landscape, and re-check the docs linked below before you commit a production design to a specific flavor.

Why it matters for your Bronze layer

- **Open format = no lock-in.** Your M&O team will ask "will BigQuery hold our data hostage?" — a fair question. The Iceberg answer: the data lives in *your* buckets as open Parquet plus open Iceberg metadata. If you ever leave, you detach the metadata and take the files; nothing was ever inside a proprietary store. And if a second engine (Spark, Trino) ever enters your estate, it reads the same files — no export pipeline, no second copy.
- **Time travel replaces a restore cycle.** Re-running a bad transformation against "yesterday's Bronze" in SQL Server means a backup/restore cycle. With Iceberg snapshots it's a query clause (`FOR SYSTEM_TIME AS OF`) or a rollback. For a team that replays Dataform releases, this is an operational superpower.
- **Governance arrives early.** BigLake tables over files in GCS support column-level security, row access policies, and dynamic data masking — meaning your Bronze can carry PII columns that analysts physically cannot select, before Dataform ever transforms a row. That is the M10 governance story arriving a zone early.

The ladder you climbed today

Three blocks, three rungs, one ladder — and the rungs are the Bronze section of your architecture doc:

1. **This morning (DIN §30):** files in a bucket became a read-only external table — a metadata pointer.
2. **The lab (13:00):** similar files got ACID table semantics through Iceberg metadata.
3. **The governance angle (M10, 14:00):** the same pattern gains fine-grained security and catalog metadata via BigLake.

For your design, the default recipe is: **land files on GCS → expose them as BigLake/Iceberg tables for queryable, governed Bronze → Dataform builds managed Silver and Gold tables on top.** Reach for BigQuery-*managed* Iceberg tables specifically when non-BigQuery engines must also write the same tables, or when your exit-the-platform story needs open metadata, not just open files. The 3-day Data Warehousing course goes deeper on that trade.

What this replaces in your current stack

Backup/restore cycles for replay and audit scenarios; staging copies made "just to be safe"; and the quiet lock-in of a proprietary storage engine — none of which survive contact with an open table format over files you own.

Test yourself

- [ ] I can explain Iceberg in one sentence: an open metadata layer that gives Parquet files in a bucket table behavior.
- [ ] I can name the three ways BigQuery touches Iceberg and say who owns the metadata in each.
- [ ] I can explain why hidden partitioning beats folder-name partitioning.
- [ ] I can explain to our M&O team why "open format" answers the lock-in question.

Dig deeper

- Apache Iceberg project site — the format itself, engine-neutral
- Apache Iceberg managed tables in BigQuery
- Introduction to BigLake tables

2.6 Workshop: Capstone — Map One of Your Real Extract Jobs

WORKSHOP **Day 2 · Tue Oct 6, 2026** Time: **15:20–15:50 CT** Length: **30 min**

Day 2 (Tue Oct 6), 3:20–3:50 PM CT — in groups of 3–4, map one real DOM DoIT extract job onto the GCP target architecture. About 20 minutes to build, then each group walks the room through theirs.

Why you're doing this

This page is backlog draft zero. When your migration project kicks off, the map your group fills in below is where the backlog starts — so photograph it and email it to yourself before you leave. Everything else in this course was preparation; this is the artifact you take back to work.

How it works

1. Form a group of 3–4.
2. Pick **one** real extract job your team owns or operates. Not the biggest one, not the simplest one — the **most typical** one. A good heuristic: the one that pages someone when it fails.
3. Fill the template below, left to right. Paper, whiteboard, or a doc — the filled table is the deliverable.
4. Every box has an answer from the last two days. If your group gets stuck on a box, don't paper over it — flag it. A box you can't fill is the most valuable thing to raise in the share-out.
5. Keep it to one job. If you catch yourselves designing a platform, come back to the one job.

The mapping template (copy it and fill it in)

Row	Your answer
The job (name it)	*e.g. nightly extract of X to Y — one real job, with its real name*
Source system	*SQL Server table(s)? File drop? SaaS? Size? Change rate?*
Ingest tool	*Datastream (CDC) / Storage Transfer / BigQuery Data Transfer Service /* `bq load` *or* `LOAD DATA` */ gcloud — pick one, and say why*
Bronze (raw landing)	*GCS files (format? partition layout?) / external table / BigLake / Iceberg — what does raw look like for this job?*
Silver (cleaned/conformed)	*Dataform models: dedupe, types, conforming — what rules does this job's data need?*
Gold (mart)	*Which mart table(s)? Partitioned/clustered by what? Who queries it?*
Orchestration & CI/CD	*Dataform release config / Cloud Scheduler / Composer — and how does it deploy? (Your existing pipeline discipline → git → release configs)*
Serving	*Power BI — Import or DirectQuery for this mart? Refresh cadence tied to what event?*
First thing that breaks	*Honest answer: volume? schema drift? PII? the source system itself?*

Decision aids (when a box stalls you)

- **Is the source a database or files?** Database → **Datastream** (log-based CDC; SQL Server is a supported source) or a batch load, depending on freshness. Files → **Storage Transfer Service** to GCS, then `LOAD DATA` with a pinned schema — remember the autodetect N/A pitfall from Day 1: pin schemas, declare null markers, fail loudly instead of loading garbage. SaaS sources → **BigQuery Data Transfer Service**.
- **Nightly is fine, or always-fresh?** Nightly tolerance → batch load, it's simpler and loads are free. "Always current" → **Datastream CDC** — and the extract window disappears entirely.
- **Import or DirectQuery?** Default to **Import**, refreshed right after the pipeline event that rebuilds the mart ("after the Dataform release") — not a wall-clock guess. DirectQuery only with a stated intra-day freshness need. The full decision table is in the Power BI connector guide.
- **Where do the transforms live?** Default: **Dataform**, in git, where your CI/CD discipline applies. Scheduled queries are fine for a single light derivation. Composer enters when there are many interdependent pipelines or non-BigQuery tasks — not on day one.

Examples to consult if you get stuck

Two filled-in maps. Peek at one if your group stalls — or finish yours first and compare afterward.

Example 1 — nightly case-records extract → Datastream CDC path

Row	Answer
The job	Nightly SQL Agent job extracts changed case records from the case-management SQL Server DB to a reporting copy; 2 AM window, occasionally collides with maintenance; downstream reports stale after missed runs
Source system	SQL Server 2019, case tables, largest ~80M rows, steady daily churn (~1–2% of rows)
Ingest tool	**Datastream** — SQL Server is a supported source; log-based CDC, no triggers on production, no extract window. Backfill + continuous changes, landing in BigQuery
Bronze	Datastream-managed BigQuery tables (append-only change events / replicated tables) = raw-as-it-gets; optionally export snapshots to GCS Parquet for the file-lake copy
Silver	Dataform incremental models: latest-record-per-case (dedupe on the change stream), type cleanup, join to county/agency reference tables
Gold	`mart_case_summary` — partitioned by case date, clustered by county; feeds the existing case-load dashboards
Orchestration & CI/CD	Dataform release config on a schedule (Silver→Gold rebuild after Datastream lands); Datastream runs itself (M&O monitors, doesn't schedule); SQLX in git → existing PR/pipeline discipline
Serving	Power BI **Import**, refresh scheduled just after the Dataform release — matches the nightly-build reality
First thing that breaks	Schema drift on the source (a new column mid-release) — Datastream handles additive changes, but the Dataform models need an assertion to catch it

Notice: the 2 AM window disappears entirely, and M&O's role shifts from "restart the failed job" to "watch a stream's freshness metric."

Example 2 — county CSV file-drop → Storage Transfer + LOAD DATA path

Row	Answer
The job	Counties drop monthly CSV extracts (case/financial summaries) to an SFTP share; an SSIS package sweeps the share, stages, cleans, loads
Source system	CSV files, ~40 counties, monthly, a few GB total; formats drift county to county; occasional late or re-sent files
Ingest tool	Files move to **GCS** (counties drop to a bucket, or **Storage Transfer Service** sweeps the existing share on a schedule); then `LOAD DATA` **SQL** with a pinned schema — never autodetect — into a Bronze table. A scheduled query with `FORMAT_DATE`-built URIs replaces the sweep
Bronze	GCS bucket, Hive layout `county=X/period=YYYY-MM/` — original files kept immutable (replayable); **BigLake table** over the files for immediate query; loaded Bronze table for the conformed copy
Silver	Dataform: per-county staging models absorbing format drift (one model per variant), unioned and typed; assertions on row counts per county per period
Gold	`mart_county_monthly` — partitioned by period, clustered by county
Orchestration & CI/CD	Cloud Scheduler → scheduled query (load) → Dataform release (Silver→Gold); the whole flow is SQL + git — no SSIS package to version
Serving	Power BI **Import**, monthly refresh after the load window (DirectQuery is pointless — the data changes monthly)
First thing that breaks	A county changing its CSV layout without telling anyone — hence pinned schemas (the load fails loudly instead of loading garbage) and a Bronze assertion on unexpected NULL rates

Notice: this is the cheapest, fastest win in your estate — no CDC infrastructure, just a bucket + `LOAD DATA` + Dataform. And "late county re-sends file" stops being a rerun-the-package event: overwrite the file in the bucket, re-run one statement.

What good looks like — check your map before the share-out

- ☐ **One tool per box, not three.** A cell that says "Datastream or DTS or Dataflow" is a decision you deferred — acceptable for draft zero, but mark it as an open question so it survives.
- ☐ **Bronze stays raw and replayable.** No cleaning rules in Bronze. Bronze answers "what did the source say"; Silver answers "what does it mean." That separation is what makes reprocessing possible — the thing your current extract jobs can't do.
- ☐ **Dataform owns Silver→Gold, and it lives in git.** If transforms ended up somewhere else on your map (heavy scheduled queries, Power Query inside Power BI), you should be able to say why — legitimate answers exist, but the default belongs in Dataform where your CI/CD discipline applies.
- ☐ **Orchestration is boring.** Cloud Scheduler + Dataform release configs covers nearly everything you mapped today. Composer appears only if you can name the interdependency that needs it.

- [ ] **Power BI is Import-first.** DirectQuery appears only next to a stated freshness need, and the Serving row names the pipeline event the refresh follows.
- [ ] **The "first thing that breaks" row is honest.** If it says "nothing breaks," you're not finished. Schema drift, PII, volume, a county that re-sends files — your real answers are the migration risk register's first entries.
- [ ] **It's specific.** Real table names, real counties, real row counts — that's what makes it backlog draft zero instead of a course exercise.

Before you leave

Photograph your map. Email it to yourself. When the migration project kicks off, this page is where the backlog starts.

KEEP-FOREVER REFERENCE

Appendices — Reference Cards

The reference cards the activities and labs point to, reproduced in full so this workbook stands on its own.

Appendix A Migration Map

Your end-to-end target architecture on one page, built up piece by piece across the two days and completed at the Day-2 capstone. This is the card to print and pin by your desk.

The whole flow

```
SOURCES                          INGEST — land it once, in one place
==============================   ================================================================
SQL Server databases       --->  Datastream: log-based CDC — backfill + continuous changes.
(case mgmt, finance, ...)        No triggers, no polling, no nightly window.
                                 SQL Server is a supported source.

County file drops          --->  Files land in a GCS bucket (Storage Transfer or scheduled copy),
(SFTP shares, counties)          then LOAD DATA with a PINNED schema — never autodetect
                                 for anything recurring.

SaaS applications          --->  BigQuery Data Transfer Service: managed, scheduled pulls.

Ad-hoc / one-off files     --->  Console upload, bq load, or LOAD DATA.
=============================================================================================================
                                  |
                                  v
BRONZE — raw, immutable, replayable
  GCS buckets: Parquet in Hive layout (county=X/period=YYYY-MM/), kept as-landed forever.
  BigLake / Iceberg tables over the files + Datastream-landed raw tables beside them.
  Queryable from minute one; the source of every future replay.
                                  |
                                  |  Dataform staging + incremental models, with assertions
                                  |  (all .sqlx in git, reviewed in PRs — your CI/CD discipline)
                                  v
SILVER — cleaned & conformed
  Deduped, typed, joined to reference data; one staging model per source variant.
  Assertions catch schema drift and surprise NULL rates before your analysts do.
                                  |
                                  |  Dataform release configuration (Silver -> Gold, on a schedule)
                                  v
GOLD — data marts, the single source of truth
  BigQuery mart tables, partitioned by date, clustered by the columns you filter on.
  Governed: IAM, Knowledge Catalog, sensitive-data protection for PII.
                                  |
                                  |  Google BigQuery connector (Get Data in Power BI)
                                  v
SERVING — the Power BI you already have
  Same reports, authors, workspaces, DAX. Import + scheduled refresh by default;
  DirectQuery only when a mart genuinely needs intra-day freshness.

ORCHESTRATION SPINE (runs the whole flow, everything defined in git):
  Cloud Scheduler (= SQL Agent)  ->  Dataform release runs  ->  Power BI refresh follows the release.
  Datastream runs itself — M&O watches a freshness metric instead of restarting a 2 AM job.
```

Stage by stage

Stage	GCP tool(s)	What it does	Replaces in your current stack
Ingest — databases	**Datastream**	Log-based change data capture: initial backfill plus continuous inserts/updates/deletes into BigQuery	Nightly extract jobs, replication

Stage	GCP tool(s)	What it does	Replaces in your current stack
Ingest — files	**GCS + Storage Transfer**, then `LOAD DATA`	Drops land in a bucket once; loaded with a pinned schema JSON	SSIS file-sweep packages
Ingest — SaaS	**BigQuery Data Transfer Service**	Managed, scheduled pulls from SaaS sources	Vendor exports + manual loads
Bronze	**GCS (Parquet, Hive layout) + BigLake/Iceberg**	Raw landing: immutable, replayable, queryable in place	Staging shares + staging tables
Silver	**Dataform** staging + incremental models, assertions	Cleaning, conforming, dedupe — with tests in the repo	Transform sprocs / SSIS data flows
Gold	**BigQuery marts** (partitioned + clustered)	Curated marts — the single source of truth for reporting	Reporting database / mart tables
Serving	**Power BI + Google BigQuery connector**	Import (default) or DirectQuery against Gold	The reports you already run
Orchestration	**Cloud Scheduler + Dataform release configs**, in git	Schedule → release → refresh, boring on purpose	SQL Agent jobs + your CI/CD pipelines
Governance	**IAM, Knowledge Catalog, Sensitive Data Protection**	One governed source of truth; PII discovered and protected	Your security & PII controls, centralized

Rules of the map

- **Every box gets one tool, not three.** A cell that says "Datastream or DTS or Dataflow" is a deferred decision — acceptable in backlog draft zero, but name it as one.
- **Bronze answers "what did the source say"; Silver answers "what does it mean."** No cleaning rules in Bronze — that separation is what makes reprocessing possible, the thing your current extract jobs can't do.
- **Dataform owns Silver→Gold, and it's in git.** Transforms living anywhere else (heavy scheduled queries, Power Query in Power BI) need a stated reason.
- **Orchestration is boring.** Scheduler + Dataform release configurations cover nearly everything you mapped in class. Composer (managed Airflow) enters when you have many interdependent pipelines or non-BigQuery tasks — not on day one.
- **Power BI is Import-first.** DirectQuery appears only with a stated freshness need, and refresh cadence names the pipeline event it follows ("after the Dataform release"), not a wall-clock guess.
- **Fill in "first thing that breaks" honestly.** Schema drift, PII, a county that re-sends files, 10× volume — that row becomes your migration risk register.

Dig deeper

- Datastream — sources (incl. SQL Server) · Storage Transfer Service · `LOAD DATA`
- Iceberg tables · BigLake · Partitioned tables · Clustered tables
- Dataform · Scheduled queries · Cloud Scheduler
- The Dataform repo walked through in class on Day 1: https://github.com/jwdavis/dataform-demo
- The last mile in detail: Power BI ↔ BigQuery connector guide

Appendix B T-SQL → BigQuery SQL

BigQuery speaks **GoogleSQL** (standard SQL). Most of your T-SQL transfers directly; this card is the differences you'll actually hit, first used on Day 1 afternoon and hardest-needed in the Day-2 Iceberg lab. Every row below matches the BigQuery docs (links at the bottom).

The everyday rows

T-SQL	BigQuery GoogleSQL	Notes
`[Order ID]`	`` `Order ID` ``	**Backticks, not square brackets.** Quote the whole three-part name as one: `` `project.dataset.table` ``
`SELECT TOP 100 *`	`SELECT * ... LIMIT 100`	`LIMIT`, **not** `TOP` — and it goes at the end of the query
`ISNULL(col, 0)`	`IFNULL(col, 0)` or `COALESCE(col, 0)`	Prefer `COALESCE` (standard, takes multiple fallbacks)
`CAST(x AS INT)` (errors on bad data)	`SAFE_CAST(x AS INT64)`	`SAFE_CAST` returns `NULL` instead of failing the query — put it in Silver-layer models that absorb vendor-file dirt
`GETDATE()`	`CURRENT_TIMESTAMP()`	Returns `TIMESTAMP` (UTC). Also `CURRENT_DATE()`, `CURRENT_DATETIME()`
`DATEADD(day, -7, d)`	`DATE_SUB(d, INTERVAL 7 DAY)`	Add with `DATE_ADD`, subtract with `DATE_SUB`; also `TIMESTAMP_ADD` / `DATETIME_ADD` / `TIMESTAMP_SUB`
`DATEDIFF(day, a, b)`	`DATE_DIFF(b, a, DAY)`	**Argument order flips**: later date first. Also `TIMESTAMP_DIFF`
`LEN(s)`	`LENGTH(s)`	
`SUBSTRING(s, 1, 3)`	`SUBSTR(s, 1, 3)`	`SUBSTRING` also works as an alias; positions are 1-based in both
`CHARINDEX('x', s)`	`STRPOS(s, 'x')`	Haystack first, needle second — the argument order flips
`s1 + s2`	`CONCAT(s1, s2)` or `s1 || s2`	`CONCAT` treats `NULL` as empty string (no `CONCAT_NULL_YIELDS_NULL` setting to remember)
`5 / 2` → `2`	`5 / 2` → `2.5`	Division always returns a floating-point type. Integer division is `DIV(5, 2)` → `2`
divide by zero → error	also an error	Use `SAFE_DIVIDE(x, y)` to get `NULL` instead

T-SQL	BigQuery GoogleSQL	Notes
`int` / `bigint`	`INT64`	One integer type. `datetime` ≈ `TIMESTAMP` (UTC) **or** `DATETIME` (civil time, no zone) — pick deliberately per column
Stored procedures, batches	**Scripting**: `DECLARE x INT64; SET x = ...;` plus `IF ... END IF;`, `WHILE ... END WHILE;`	Multi-statement scripts run straight in the query editor — your "anonymous sproc" instinct works
`MERGE`	`MERGE`	Exists, same purpose: upsert by match condition
CTEs (`WITH`), window functions (`ROW_NUMBER() OVER (PARTITION BY ...)`)	identical	Work exactly as you expect
`USE mydb`	—	**No** `USE`**.** Always fully qualify: `` `project.dataset.table` ``

Things that will bite you

1. **Case sensitivity.** String comparisons are case-sensitive (`'smith' ≠ 'Smith'`), and dataset and table names are case-sensitive by default. If your SQL Server collation is the usual case-insensitive sort, this is the difference most likely to produce "the same" rows not joining.
2. **Project IDs contain dashes — backtick the whole name.** Lab projects have IDs like `qwiklabs-gcp-01-ab12cd34ef56`. `` `qwiklabs-gcp-01-ab12cd34ef56.class.orders` `` works; unquoted or partially quoted versions fail.
3. **Integer math doesn't truncate, and zero divides loudly.** `/` returns a float; divide-by-zero errors out the whole query. Reach for `DIV()` and `SAFE_DIVIDE()`.
4. **Nested/repeated columns need** `UNNEST`**.** BigQuery tables can hold `ARRAY<STRUCT<...>>` columns (an order with its line items inside the row). You can't treat them like a child table until you flatten: `FROM orders, UNNEST(line_items) AS li`. You'll meet this in the derived-tables activity (DIN §29).
5. `TIMESTAMP` **≠** `DATETIME`**.** Comparing or joining the two types errors instead of implicitly converting. Decide per column: `TIMESTAMP` for event instants, `DATETIME` for "wall-clock" values like a business date.
6. **External-table DDL is metadata-only and returns instantly.** If a `CREATE EXTERNAL TABLE` seems to hang, the statement is wrong (usually the URI) — it's not "reading the files."

Pleasant surprises

- `SELECT * EXCEPT (cust_email, cust_phone)` — project everything *but* the PII, no column list.
- `SELECT * REPLACE (SAFE_CAST(minutes AS INT64) AS minutes)` — re-type a column inline.
- `QUALIFY` — filter on a window function without a subquery: `... QUALIFY ROW_NUMBER() OVER (PARTITION BY case_id ORDER BY updated_at DESC) = 1` — your "latest record per key" pattern in one line.

Dig deeper

- Query syntax (identifiers, `LIMIT`, CTEs, `UNNEST`, `QUALIFY`)
- Conditional expressions (`IFNULL`, `COALESCE`)
- Conversion functions (`SAFE_CAST`)
- Date functions · Timestamp functions · String functions · Operators (`DIV`, `SAFE_DIVIDE`)
- Scripting (`DECLARE`, `SET`, `IF`, `WHILE`)
- DML (`MERGE`)

Appendix C BigQuery Console Survival Guide

Written for your first hours in the Google Cloud console — Day 1 at 10:10, then every lab after. Everything here is clickable in your lab project; nothing here can break anything.

The project picker (check it first, every time)

The bar at the very top of the console shows the **currently selected project** next to the Google Cloud logo. Every page you open shows resources for *that* project only.

> **Gotcha — the wrong-project trap:** if the console looks empty, a dataset you just made has "disappeared," or a lab step's screenshots don't match what you see, you're probably in the wrong project. Click the project name in the top bar and switch. In this course your labs hand you one project each — if the picker shows something else, fix that before anything else.

Starring (pinning) projects

Your own project appears in the BigQuery **Explorer** pane automatically. To see *other people's* projects — the public datasets and demo data we use all course — you star them:

1. Open **BigQuery** (hamburger menu → BigQuery, under Analytics — or search "BigQuery" in the top search bar).
2. In the **Explorer** pane, click **+ ADD** → **Star a project by name**.
3. Enter the project name, e.g. `bigquery-public-data`, and click **STAR**.

On Day 1 you star three: `bigquery-samples`, `bigquery-public-data`, `roi-bq-demos`. Starred projects are read-only windows into someone else's data — you query across projects without copying anything. (Docs)

The Explorer pane

The left-hand tree is **project** → **dataset** → **table** — your instance → database → schema.table instinct applies, minus the schema layer.

- Expand a dataset to see its tables and views.
- The **three-dot menu** next to any project, dataset, or table is where the actions hide: **Create dataset**, **Create table**, open, delete, and so on. When you're stuck, look for a three-dot menu.

Schema / Details / Preview — the three tabs that save you money

Click any table and three tabs open:

- **Schema** — column names, types, modes. The first place you look in a new dataset.
- **Details** — row count, size, location, labels, last modified.
- **Preview** — a free peek at actual rows, *without running a query and without scanning bytes*.

> **Gotcha:** `SELECT * FROM big_table LIMIT 10` bills a scan of the columns it reads; the Preview tab shows you rows for free. Use Preview to look, use queries to compute.

The query editor

- Open a new query tab with the **+** ("Compose new query") at the top of the editor area.
- **RUN** executes. Right beside it, the editor shows a **bytes-processed estimate** ("This query will process N when run") — the single most important habit of the course: **read that number before you press RUN**, especially against public datasets with billions of rows. A 100-billion-row table is free to browse and preview, expensive to `SELECT *`.
- **RUN ≈ executing in SSMS; the estimate ≈ glancing at the execution plan first.** Same instinct you already have.
- **Query history:** the bottom panel's **Personal history** / **Project history** lists every job you've run — click one to re-open its SQL. Nothing you ran is lost; you can always get yesterday's query back.
- Worth memorizing: **Ctrl+Enter** (Windows) / **Cmd+Enter** (Mac) runs the query. BigQuery Studio has a built-in keyboard-shortcut reference for the rest; the full web-UI tour is in the docs.

Cloud Shell

The **terminal icon (`>_`)** at the top right opens Cloud Shell: a small temporary VM in your browser with `gcloud` and the `bq` CLI preinstalled and already authenticated as you.

- **First launch:** it asks you to click **Continue** to start, and may show an **Authorize** consent dialog before running commands. Clicking through these is normal and expected — everyone does it once per environment.
- The `bq` command line is the workhorse for scripting loads: `bq load`, `bq show --schema`, `bq query`. The Day-1 loading block uses it; the files to paste are in your Day-1 activity companions.
- If Cloud Shell asks which project to use, pick your lab project (same wrong-project trap, different room).

"Enable API?" — not an error

The first time you open certain services in a fresh project (Datastream, Dataform, Cloud Scheduler, …), the console may offer an **Enable API** button. That's the platform turning that service on for your project — click **Enable**, give it a moment, continue. In labs this is routine, not a problem to report.

Where IAM and Billing live

- **Hamburger menu → IAM & Admin → IAM** — who (people, groups, service accounts) can do what on this project. This is where the service-account discipline from your SQL Agent proxy world lands.
- **Hamburger menu → Billing** — what this project costs. In labs, billing is handled for you; back home, this is where your FinOps questions get answered.

You won't administer either in class — but know where they are. (IAM & service accounts)

If you get lost

- **The search bar at the top of the console finds everything** — services, pages, settings. Type "BigQuery", "Storage", "Datastream", "Scheduler".
- **Hamburger menu → BigQuery** returns you to BigQuery Studio from anywhere.
- Bookmark **https://console.cloud.google.com/bigquery** — it goes straight to the studio.
- Still lost? Say so. Sixteen people navigating one new console — someone else is lost in the same place.

Appendix D GCP for SQL Server Professionals

The big mental-model card: your SQL Server world on the left, the GCP equivalent on the right. Read it once before Day 1, keep it open during labs, and let the right-hand column become reflexive by Tuesday afternoon.

One caution before the table: these are *mappings*, not identities. A GCP project behaves like an instance in the ways that matter to you (boundary of access and billing) and differently in ways that don't (nobody patches it). Where the difference bites, the notes column says so.

Compute & structure

Your SQL Server world	GCP world	Notes
SQL Server instance	**GCP project**	The top-level container: resources, permissions, and billing live at the project boundary. Nothing to install, patch, or fail over.
Database	**BigQuery dataset**	Container for tables and views. You choose its **location** (region/multi-region) at creation and can't change it later — decide where your warehouse lives once, deliberately.
`schema.table`	`dataset.table`	BigQuery has no schema layer. The fully qualified name is `` `project.dataset.table` `` — three parts, backticks.
Sizing the instance (cores, RAM)	**Nothing to size**	BigQuery is serverless: compute is automatic per query. Capacity planning only re-enters if you later choose flat-rate **slots** instead of on-demand pricing.

Storage

Your SQL Server world	GCP world	Notes
MDF/NDF data files	**BigQuery managed storage**	You never see, place, or manage storage files — you create tables, storage follows.
File shares / SFTP drops	**GCS buckets** (Cloud Storage)	Object storage. "Folders" are naming conventions in object keys, not real directories — but the Hive layout (`county=X/period=YYYY-MM/`) buys you real partition pruning.
Staging share for extracts	**A GCS bucket (your Bronze layer)**	Files land once, stay immutable, and are queryable in place via external/BigLake/Iceberg tables. No staging-server disk to babysit.
Backup & restore	**Time travel + table snapshots**	Query any table as it was up to 7 days ago (default window); snapshots for longer. GCS versioning covers the bucket side.

Security

Your SQL Server world	GCP world	Notes
Logins & users	**Google identities + IAM**	Access is granted to people, groups, or service accounts — on projects, datasets, or individual tables.
SQL Agent proxies / job accounts	**Service accounts**	Non-human identities for pipelines. The least-privilege discipline you already apply transfers one-for-one: a refresh identity that may only read the Gold dataset and run queries.
`db_datareader` / `db_datawriter`	**IAM roles** (e.g., BigQuery Data Viewer, Job User)	Roles bundle permissions; scope them as tight as you scope database roles today.
Row/column-level security, dynamic masking	**Row access policies, column-level security, policy tags**	Exists in BigQuery — covered in the Day-2 governance block.

Operations & orchestration

Your SQL Server world	GCP world	Notes
SSIS packages	**Dataform** (ELT inside BigQuery) / **Dataflow** (heavy or streaming transforms)	Most of your transforms become plain SQL in Dataform models. Dataflow is the escape hatch when SQL genuinely isn't enough.
SQL Server Agent jobs	**Cloud Scheduler** / **scheduled queries**	A SQL statement on a schedule — you can build one with a few clicks, and you will on Day 1.
Linked servers / `OPENQUERY`	**Federated queries** (connection resource + `EXTERNAL_QUERY`) / **BigLake** for files	The connection resource is the linked-server definition; `EXTERNAL_QUERY` is `OPENQUERY` — including the "inner SQL is the remote dialect" trap.
Replication / log shipping / CDC	**Datastream**	Log-based CDC from SQL Server (and Oracle/MySQL/PostgreSQL) into BigQuery: backfill plus continuous changes, no triggers, no nightly window.
Windows Task Scheduler	**Cloud Scheduler**	Managed cron for anything on a timer.
SSIS projects in source control	**Dataform repositories in git**	`.sqlx` files, branches, pull requests, release configurations — the shape of the CI/CD pipelines you already run.
Agent job history / DMVs	**Job history,** `INFORMATION_SCHEMA` **views, Cloud Monitoring**	Every query and load is a job with metadata you can query in SQL.

Instincts that transfer

You don't need new engineering judgment for this platform — you need to point your existing judgment at new consoles. Seven instincts, unchanged:

1. **Your CI/CD discipline is exactly how Dataform releases work.** Models are `.sqlx` files in git; you review changes in PRs; a release configuration promotes a git commit to run on a schedule. The pipeline rigor your team built for SQL Server deployments is the correct mental model, and most teams new to GCP don't have it — you do.

2. **You never trusted "Suggest Types" in SSIS — don't trust autodetect here.** Autodetect samples the start of a file; a dirty value deeper in gets either a loud failure or a silently wrong schema (and the silent one costs you days). For anything recurring: pin a schema JSON in source control and name your null markers (`--null_marker='N/A'`). You do this dance on Day 1.

3. **Jobs should be idempotent and re-runnable.** Same rule, new syntax: overwrite partitions (`WRITE_TRUNCATE`), use incremental models keyed on a watermark, and design so that re-running last Tuesday is a non-event.

4. **Least-privilege service identities.** A job gets a service account scoped to exactly what it reads and writes — the proxy-account discipline, one-for-one.

5. **Environment separation.** Dev/test/prod become separate GCP projects (or datasets), with IAM doing the walling-off you used to do with separate instances.

6. **Estimate before you execute.** You read execution plans before running something big in production; here you read the **bytes-processed estimate** next to the RUN button before pressing it. Same instinct, cheaper mistakes.

7. **Monitor freshness, not job success.** A replication stream has a freshness metric instead of a 2 AM job that either mailed you or didn't. The M&O question shifts from "did the job finish?" to "how stale is the data?" — a better question, and one you already know how to operationalize.

Where this card gets used

- Day 1, 09:25 — the building-blocks block walks the compute & storage rows
- Day 1, 11:10 — the replication row becomes your extract-killer
- Day 1, 16:05 — the orchestration rows become your automation map
- Day 2, 14:00 — the security rows become governance across the medallion
- Always: the glossary defines any term above in one line

Appendix E Power BI ↔ BigQuery Connector Guide

The last-mile guide from Day 2 (Tue Oct 6, ~3:05 PM CT), written to keep: how to point your existing Power BI reports at BigQuery Gold marts — desktop connection, the Import vs DirectQuery decision, service-side credentials, and the gateway answer for the M&O team.

Why this matters for your migration

Your migration's endgame is one sentence: **your reports don't change, their source does.** The state's Power BI estate stays exactly as it is — same report authors, same workspaces, same consumers. The only thing that changes is where "Get Data" points. This guide is that change, end to end.

Before you start

- **Power BI Desktop, June 2021 or newer.** Google sign-in from Power BI requires it (Google discontinued embedded-browser sign-ins in 2021).
- **Your own Google account**, with access to the GCP project that holds the Gold dataset. To connect and query, your account needs `roles/bigquery.dataViewer` on the Gold dataset and `roles/bigquery.jobUser` on the project — your GCP admin grants both. If you can query the mart in the BigQuery console, you can connect to it from Power BI.
- **Know your target's full name:** project → dataset → table — the same hierarchy you used in the console all course long.
- Reference doc to bookmark: Microsoft keeps the connector's supported-features table and auth notes at Google BigQuery connector (Power Query). Check it when you set this up for real — the details below are stable, the dialogs evolve.

Step 1: Connect from Power BI Desktop

1. **Home** → **Get Data** → **More…**, then find **Google BigQuery** (search for it, or look in the Database category) → **Connect**. The connector ships in the box; it is the same Power Query connector Excel uses.
2. Leave **Advanced options** alone for a first connection (that is where a billing project or a custom SQL statement would go). Select **OK**.
3. **Sign in.** Choose the organizational-account sign-in, then **Sign In**. A "Sign in with Google" dialog appears: pick your Google account and approve connecting to Power BI Desktop. Select **Connect**.
4. **Navigator.** Expand the tree — project → dataset → table — and pick your mart table. Projects and datasets appear exactly as they do in the GCP console: the resource hierarchy you learned on Day 1, surfacing inside your existing BI tool.
5. **Choose Import or DirectQuery** when the dialog offers both. For a first look, pick **Import** and load — the decision deserves the table below, not a coin flip.
6. Select **Load**. The table lands in the model, in the Fields pane. From here everything is the Power BI you already know — DAX, visuals, publish to the service. Nothing on the report side changes.

What the Google sign-in actually means: the connector uses OAuth, so **BigQuery sees the report author as a person.** IAM on the Gold dataset applies to each author individually — including any row- or column-level security you define there — and the BigQuery audit trail shows real people running real queries, not a shared SQL login. Access is granted and revoked in IAM, per person. That is a feature: treat it as one, and never share a single Google account across authors.

Step 2: The decision — Import vs DirectQuery

The connector supports both. They are not interchangeable; choose per mart.

	Import (data cached in Power BI)	DirectQuery (live queries to BigQuery)
Latency	Stale until the next refresh — fine for daily marts	Current on every visual interaction
BigQuery cost	One query per refresh — small and predictable	A query per visual, per user, per click — bills scan bytes continuously; on on-demand pricing this needs watching
Report performance	Fastest — data sits in-memory in Power BI	Bounded by BigQuery response time — usually seconds; fine on well-partitioned Gold tables
Features	Full DAX, all visuals	Some DAX and visual limitations (standard DirectQuery rules)
Fits Iowa when…	**Default for your nightly-built Gold marts** — Dataform rebuilds the marts on a schedule anyway, so a refresh right after the build matches reality	Executives need intra-day freshness, or a dataset is too large to cache

Recommendation: start with Import plus scheduled refresh, aligned to the Dataform release schedule. Your marts are rebuilt nightly by design — DirectQuery buys freshness the pipeline does not have yet. Revisit per mart when streaming use cases appear; and note the cost math changes if you move to slot/flat-rate pricing — that is a Data Warehousing course conversation.

Step 3: Publish — auth in the Power BI service

Two identities, two stories. Keep them distinct.

- **Power BI Desktop (report authors):** each author signs in with their **own Google account (OAuth)**, as above. BigQuery sees the person; IAM and row/column security apply per author; the audit trail is real people. No service accounts on desktops.
- **Power BI service (published reports, scheduled refresh):** the stored data-source credentials should be a **service account** — a non-human identity whose only job is "read the Gold dataset." Minimum roles: `roles/bigquery.dataViewer` on the Gold dataset, plus `roles/bigquery.jobUser` on the project so it can run query jobs. Set this under the semantic model's **Settings → Data source credentials** in the service; the connector supports service-account sign-in (account email plus JSON key). Scheduled refresh of an Import dataset runs as this identity.

You already enforce this discipline today — a service account for the Power BI service is the same instinct as a SQL Agent proxy account: a least-privilege, non-human identity that owns exactly one job. Same rule, new console.

One honest caveat: the exact credential options in the service evolve. The pattern — user OAuth on the desktop, service account in the service — is stable; confirm the current dialogs on the Microsoft connector page when you set it up.

Gateway note — for the M&O team

BigQuery is a **cloud** source: scheduled refresh from the Power BI service to BigQuery needs **no on-premises data gateway**. The gateway enters the picture only if a report **mashes up BigQuery with on-prem sources** (your SQL Server, file shares) — then one on-premises data gateway serves the on-prem legs, the same gateway infrastructure you run today. As sources move to the cloud, your gateway footprint gets quieter, not bigger.

Common gotchas

- **Signed in with the wrong Google account.** Lab accounts, personal Gmail, work identity — the OAuth dialog happily uses whichever browser session is active, and Power BI caches it. Symptom: Navigator shows the wrong projects, or none. Fix: **File → Options and settings → Data source settings**, clear the BigQuery permission, and sign in again deliberately.
- **Can't see your project or dataset in the Navigator.** That is almost always IAM, not the connector: your account needs `dataViewer` on the dataset and `jobUser` on the project. Test in the BigQuery console first — if you can query the table there, the connector will see it.
- **Wrong project picked.** Same wrong-project trap as the console picker on Day 1 — in an estate with many projects, confirm the project name before you expand anything.
- **DirectQuery bill shock.** Every visual interaction by every user issues a live BigQuery query. On on-demand pricing, a popular DirectQuery dashboard is a metering device. This is why the default above is Import.
- **Scheduled refresh fails after publish.** You loaded data in Desktop, so it must be fine — no: the service needs its own stored credentials (the service account above), and that account needs `jobUser` or every refresh job fails to run.
- **Google sign-in fails or loops.** Check the Power BI Desktop version — pre-June-2021 builds cannot complete Google sign-in.
- **IT policy demands a gateway.** Only true if the report also touches on-prem sources. BigQuery alone never needs one.

Verify it works

- [ ] The mart table appears in the Fields pane after Load, with the expected row count (compare against a `COUNT(*)` in the BigQuery console).
- [ ] A visual you already own (reuse an existing report page) renders against the new source with no measure changes.

☐ After publish, the semantic model's data-source credentials hold the service account, and a manual **Refresh now** succeeds in the service.

☐ BigQuery's job history shows your queries under your identity (desktop) and the service account's identity (service refresh) — the audit story you will be asked about.

What this replaces in your current stack

- The SSIS extract → reporting-copy tables that exist only so Power BI has something stable to read — Gold marts in BigQuery take that role.
- The ODBC DSN / SQL Server connection entries in every author's Power BI Desktop — replaced by the Google BigQuery connector and per-person OAuth.
- The shared SQL login embedded in published datasets — replaced by a least-privilege service account, with real people in the audit trail.

The closing idea

> Your Gold marts land in the Power BI you already have. Same report authors, same workspaces, same DAX, same consumers — the only thing that changes is the connection string. Nothing on the report side changes. That is the whole migration, end to end: kill the extracts, land in BigQuery, Bronze/Silver/Gold with Dataform, and the reports keep working.

Appendix F Glossary

One line per term, in the order you'll meet them. When a word flies past in class, look it up here and keep moving.

The platform

- **GCP project** — the top-level container for resources, permissions, and billing; loosely your SQL Server instance's scope, with nothing to patch.
- **BigQuery** — Google Cloud's serverless data warehouse: tables and SQL at any scale, no instances to size; the engine your warehouse migrates to.
- **Dataset** — BigQuery's container for tables and views (≈ a database); lives in a fixed location you choose at creation.
- **GCS (Google Cloud Storage)** — object storage; the file share of GCP and the physical home of your Bronze layer.
- **Bucket** — a GCS container for files (≈ a share root); "folders" are just naming conventions in object keys.
- **Slot** — BigQuery's unit of compute capacity; invisible on on-demand pricing, reserved in blocks on flat-rate.

Storage & formats

- **Parquet** — compressed columnar file format; loads and scans dramatically faster than CSV (the Day-1 format race makes this felt).
- **External table** — a read-only BigQuery table that's only a metadata pointer at files in GCS (or another source); the file is the table, no load ever ran.
- **BigLake** — BigQuery tables over open-format files in GCS, adding table semantics (metadata, governance, security) to the lake.
- **Apache Iceberg** — an open table format on object storage that gives raw files snapshots, schema evolution, and ACID-ish table behavior; your Bronze/Silver candidate.
- **Medallion architecture** — the Bronze → Silver → Gold layering of raw, conformed, and curated data; the pattern in your own architecture doc, and in Google's slides.
- **Bronze / Silver / Gold** — raw-as-landed and replayable / cleaned and conformed / curated marts; the separation is what makes reprocessing possible.

Moving data

- **CDC (change data capture)** — streaming inserts/updates/deletes from the source's transaction log instead of re-extracting whole tables nightly.
- **Datastream** — Google Cloud's managed CDC service: log-based replication from SQL Server, Oracle, MySQL, or PostgreSQL into BigQuery; the extract-killer.
- **Storage Transfer Service** — managed, scheduled movement of files into GCS from shares, other clouds, or other buckets.

- **BigQuery Data Transfer Service (DTS)** — managed, scheduled pulls from SaaS apps (and Google sources) straight into BigQuery tables.
- **Pub/Sub** — managed messaging/queueing (think Service Broker at cloud scale); the front door for streaming data.
- **Federated query** — querying a live external database from BigQuery SQL via a connection resource and `EXTERNAL_QUERY`; ≈ linked server + `OPENQUERY`.

Transform, schedule, orchestrate

- **ELT** — extract, load, *then* transform: land raw first, transform inside the warehouse with SQL; your SSIS transforms move into Dataform models.
- **Dataform** — SQL workflow tooling for ELT inside BigQuery: models, dependencies, assertions, releases, all in git; your Silver→Gold engine, and where your CI/CD discipline lands.
- **Cloud Scheduler** — managed cron (≈ SQL Server Agent schedules); triggers queries, Dataform runs, and anything else on a timer.
- **Scheduled query** — a SQL statement saved to run on a schedule inside BigQuery; the cheapest automation on the platform (you build one on Day 1).
- **Composer** — managed Apache Airflow for complex, multi-system orchestration; more than you need on day one.
- **Dataflow** — managed Apache Beam for heavy or streaming transforms when SQL genuinely isn't enough; the escape hatch, not the default.

Security & access

- **IAM (Identity and Access Management)** — who (people, groups, service accounts) can do what on which resource; applies from project down to dataset and table.
- **Service account** — a non-human Google identity for jobs and apps; ≈ your SQL Agent proxy accounts, same least-privilege discipline.
- **OAuth** — the "sign in with Google" consent flow; how a Power BI Desktop author authenticates to BigQuery as themselves, so IAM sees a real person.
- **Knowledge Catalog (Dataplex)** — Google Cloud's data catalog and governance layer: discovery, metadata, and lineage across lakes and warehouses; the console and docs may still say Dataplex.

Serving & cost

- **DirectQuery** — a Power BI storage mode that queries the source live on every interaction; against BigQuery it bills scan bytes continuously, which is why Import is your default.
- **On-demand vs slots pricing** — BigQuery's two compute billing models: pay per byte scanned per query (on-demand) or reserve capacity flat-rate (slots/editions); start on-demand, revisit in the data-warehousing course.
- **BigQuery ML (BQML)** — training and serving ML models with SQL inside BigQuery (`CREATE MODEL` → `ML.PREDICT`); a Day-2 teaser, not your day-one path.

Field Notes — lessons from running the labs, recorded live

Imran Ahmad · recorded October 4, 2026, in the live console Supplement to the course WorkBook — one page of "what actually happened" per recorded walkthrough, with the screenshots to prove it. Everything below was done for real, on camera, the same week as delivery: the Day-2 federated-query lab dry-run on the trainer tenant, and the Day-1 activities (console orientation, loading block, closing scheduled query) in the instructor's own project. Where the guide and the live console disagreed, the live console won — those deltas are flagged for class.

---

1. Federated Query with BigQuery and AlloyDB (Day-2 lab)

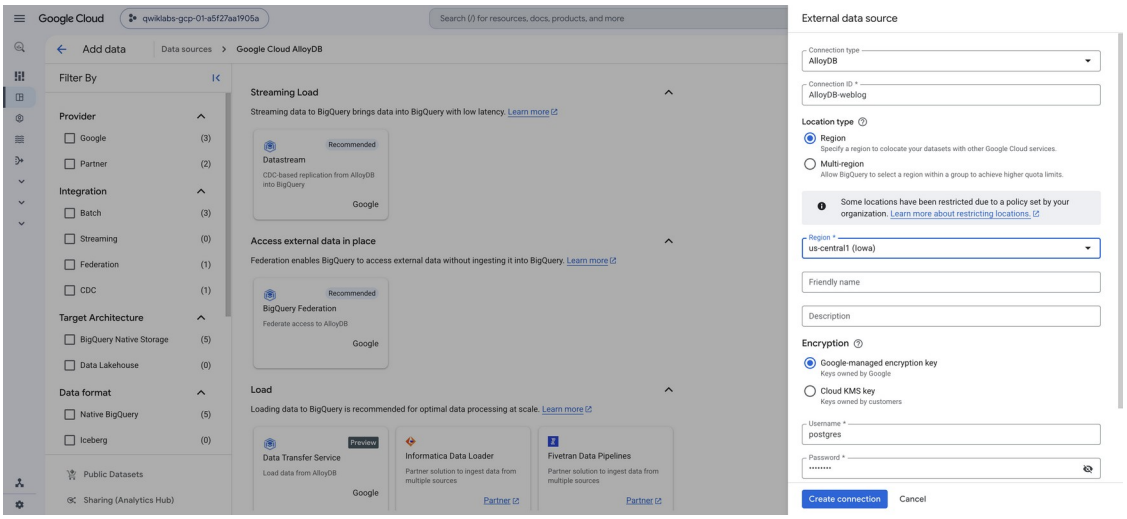
The one-sentence summary: federation is a linked server plus OPENQUERY — but two setup traps will eat your class time if you don't pre-empt them, and neither is in the lab manual.

Lesson 1 — the manual's values are placeholders; the real password is per-lab

The static lab page shows postgres as the database password. The **running** lab substitutes a real one — and in this lab it equals the console password from the lab panel. A connection created with the placeholder fails with:

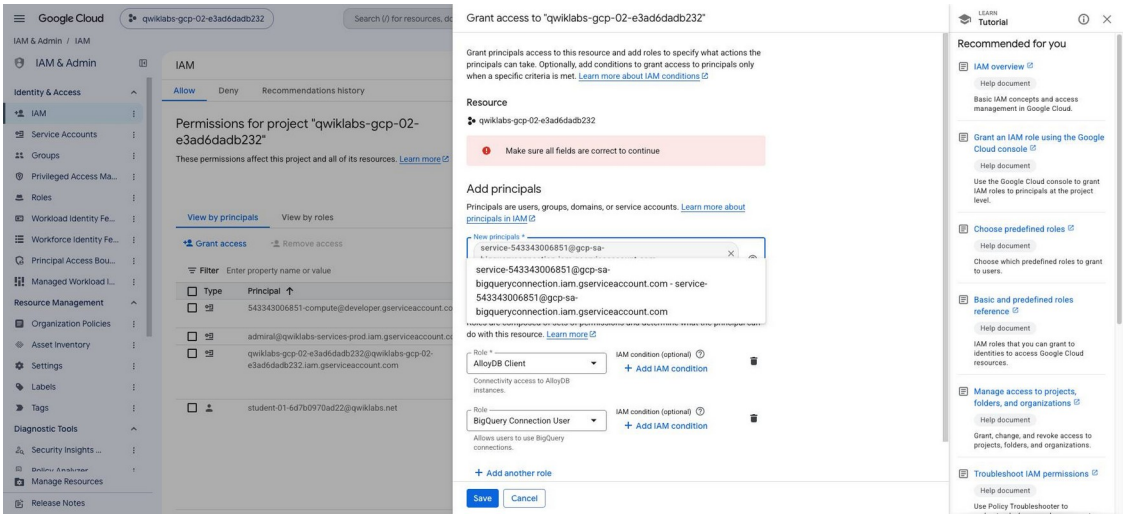
> Invalid table-valued function EXTERNAL_QUERY — Connect to PostgreSQL server failed: server closed the connection unexpectedly

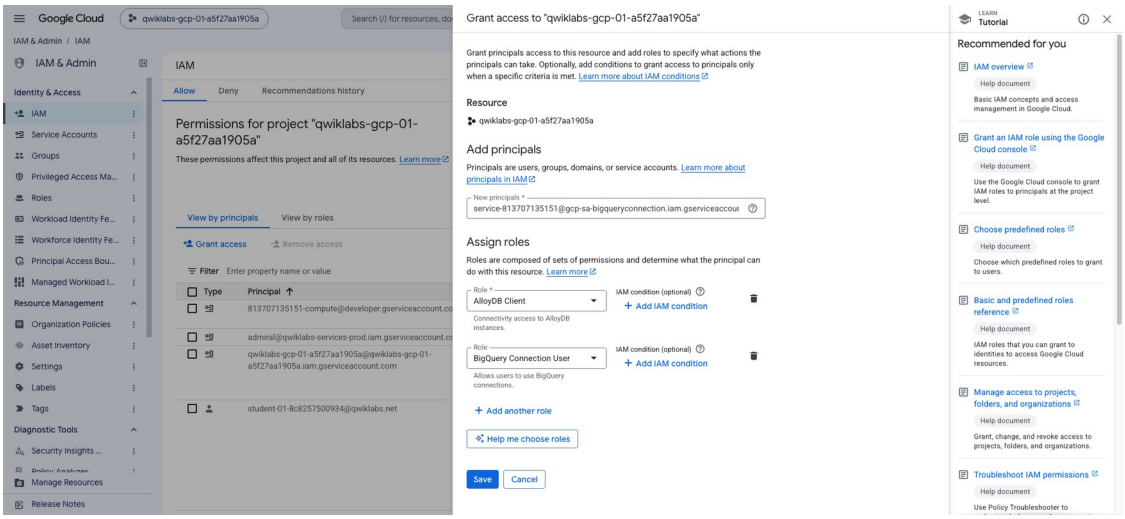
That message does not say "bad credentials" — it looks like a server crash. Google's own troubleshooting page for this exact error lists two causes: invalid credentials, and the connection service account missing **AlloyDB Client**. Both were confirmed live (see lesson 2). **Tell students: copy the connection form values from YOUR running lab's instruction panel, never from a screenshot or a stale manual page.**



Lesson 2 — the IAM chip trap (the #1 silent failure)

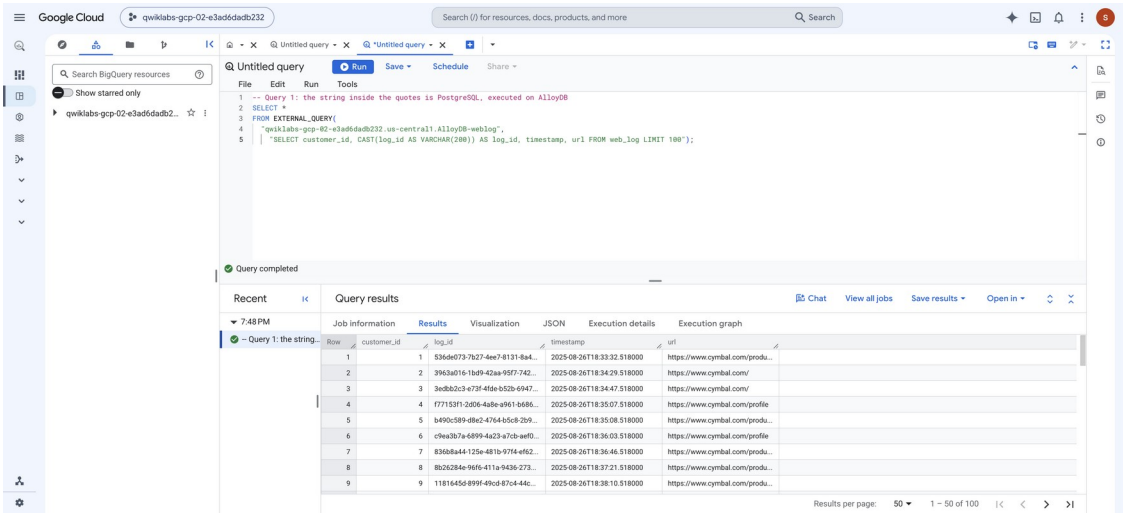
Granting the connection's service account its roles looks trivial, and silently does nothing if you miss one keystroke. Pasting the SA into **New principals** is not enough — you must **press Enter** so the address becomes a *chip*. Without the chip, Save is rejected with "Enter at least one principal" — easy to miss because the email is sitting right there in the box. The grant only counts when the **'Policy updated'** toast appears and the SA shows up in the IAM list. Without this grant, every EXTERNAL_QUERY fails with the same "server closed the connection" error as lesson 1 — two different root causes, one misleading message.



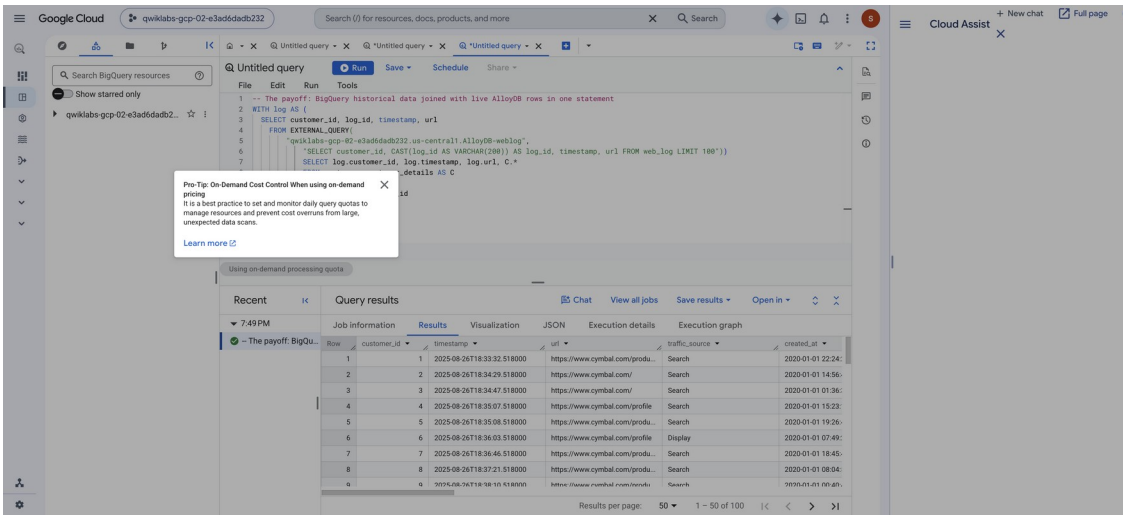


What success looks like

First EXTERNAL_QUERY — 100 rows from AlloyDB's web_log, answered from inside BigQuery:



The payoff — one statement joining BigQuery's customers table with the live AlloyDB rows:



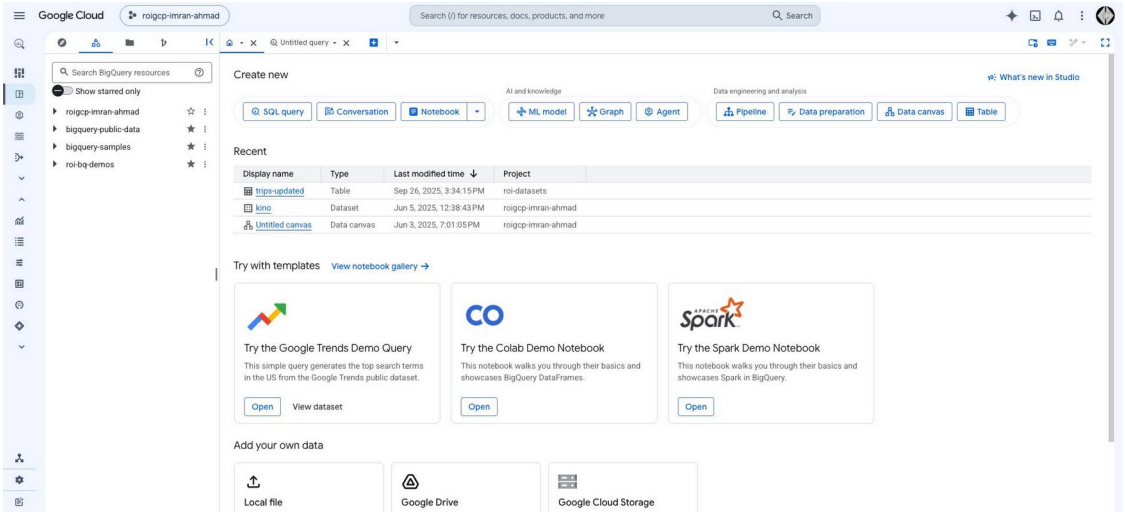
Ops note for class: the 45-minute lab timer is hard, and the tenant killed one lab account mid-run during the dry-run. Work briskly, run the progress checks early, and never leave verification to the last five minutes.

---

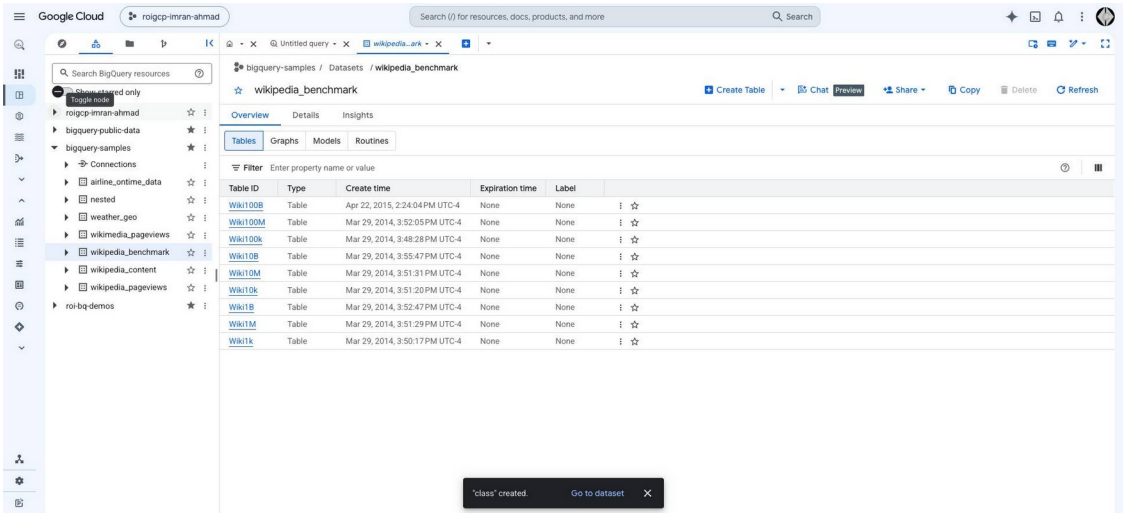
2. Console Orientation — projects, datasets, tables (Day-1 activity 03)

The one-sentence summary: the console moved again — starring is no longer "+ Add → Star a project by name"; you open the project and click its star. Everything else in the activity behaved as written.

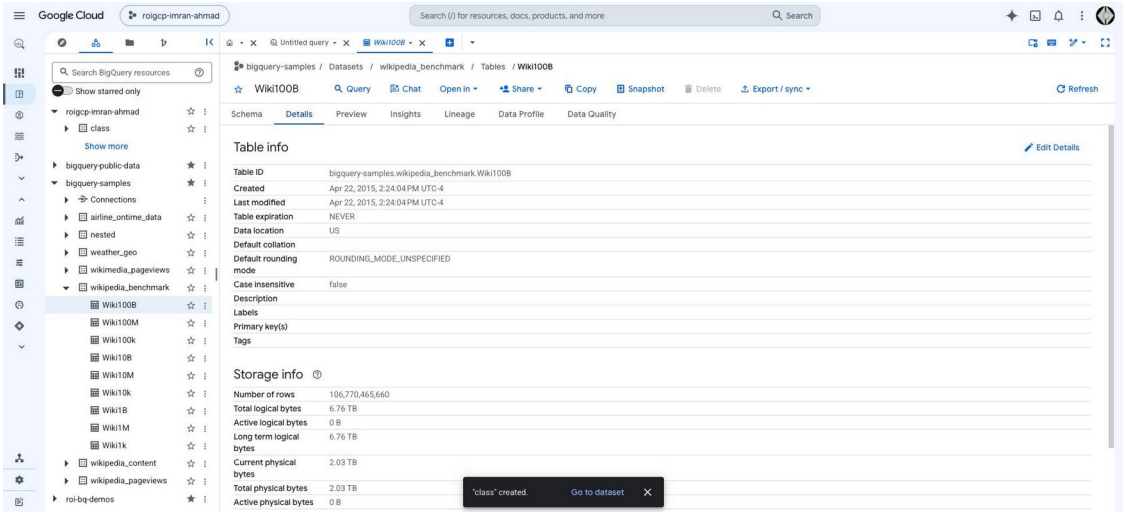
- **Starring flow changed.** The old "+ Add" entry point is gone from the current console. What works today: open the project (direct URL or the project picker), then click the **star outline** next to its name in the Explorer. The activity guide was updated to match.
- **Project picker discipline is the #1 confusion killer.** The console happily shows your last-used project; "I can't see what the instructor sees" is always the picker first.
- **Starred projects are read-only** — permission errors inside bigquery-samples, bigquery-public-data, or roi-bq-demos are expected, not a bug.

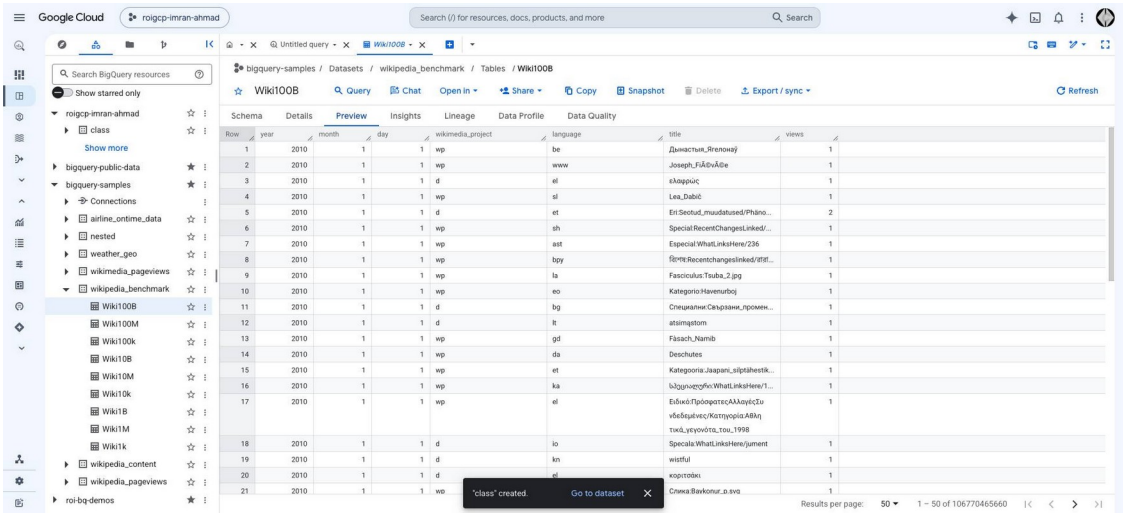


- **The class dataset is the afternoon's landing zone** — created in one dialog, nothing to size.



- **The scale story, felt.** The Wiki100B table: **106,770,465,660 rows, 6.76 TB logical** — and the Preview tab browses it with zero query, zero cost.





---

3. Loading Block — four ways to land files (Day-1 activity 08, DIN §24–§27)

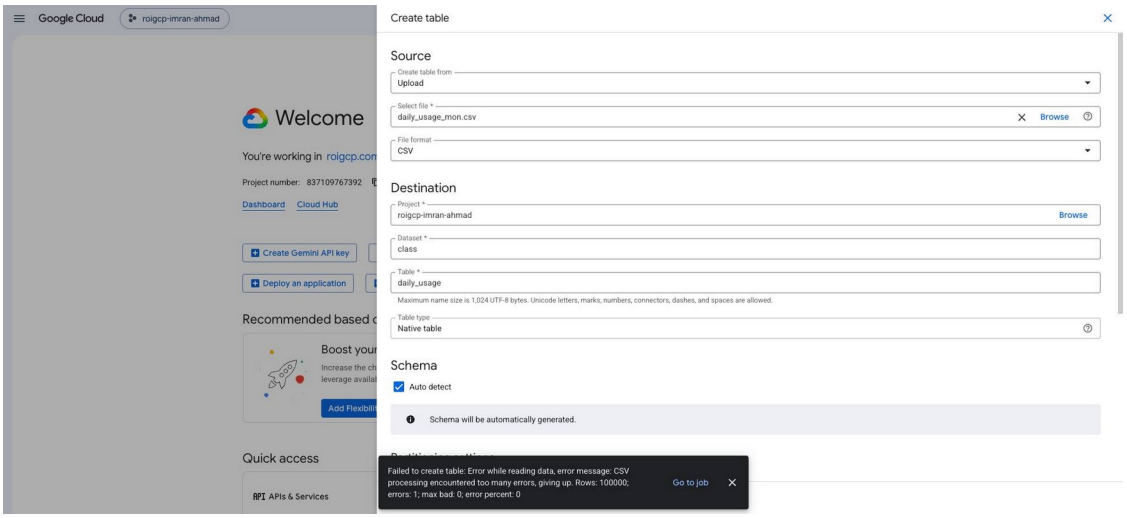
The one-sentence summary: console, CLI, and SQL are one machine; autodetect fails two ways and the quiet one is the dangerous one; pin your schemas — on appends too; and Parquet embarrasses gzipped CSV.

Console, CLI, SQL — same machinery

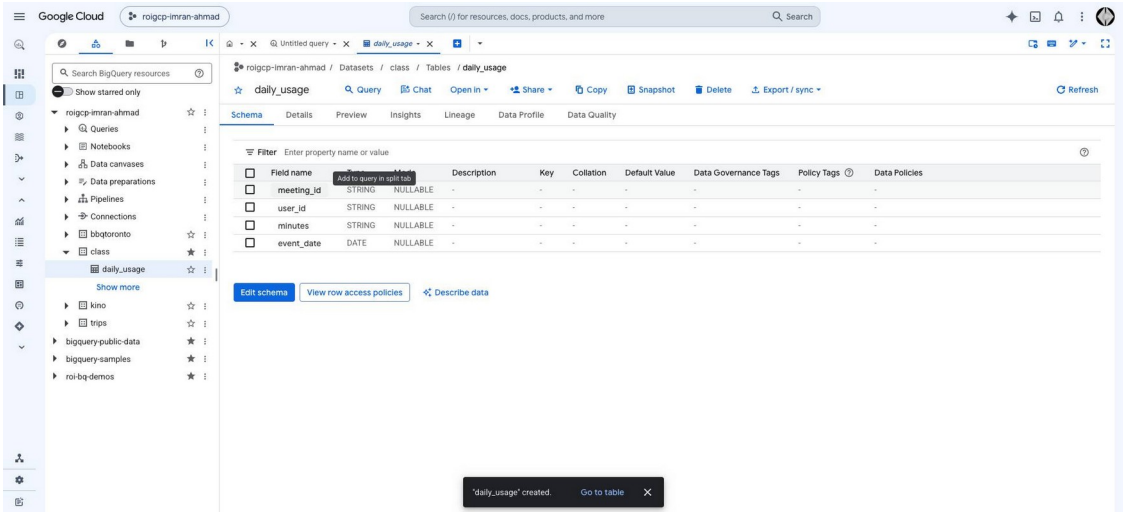
The console **Create table** load and the `bq load` append produced two identical **Load** jobs in the job history (and the `LOAD DATA` SQL statements produced the same job type again). All batch loads billed **0 B** — free. The only thing that ever billed in the whole block was a verification query.

Autodetect fails loudly once, silently once

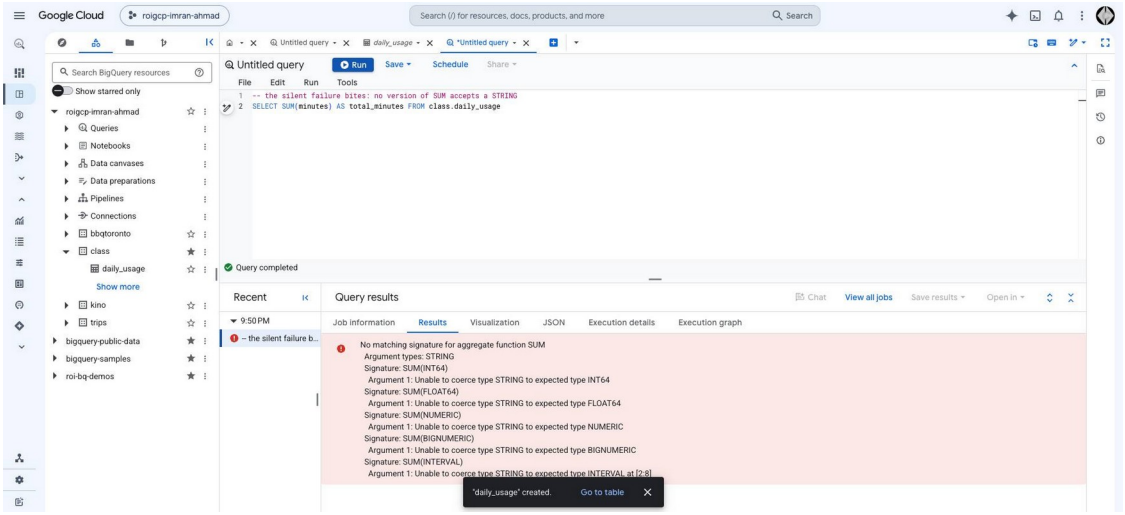
Monday's file (100,000 rows, N/A at row 90,000): autodetect sampled the top, chose INTEGER for minutes, and the load **failed** — "CSV processing encountered too many errors… Rows: 100 000; errors: 1." Loud, annoying, *safe*.



Tuesday's file (1,000 rows, N/A at row 700): the N/A fell **inside** the sample, autodetect picked the only type that fits — **STRING** — and the load "succeeded."

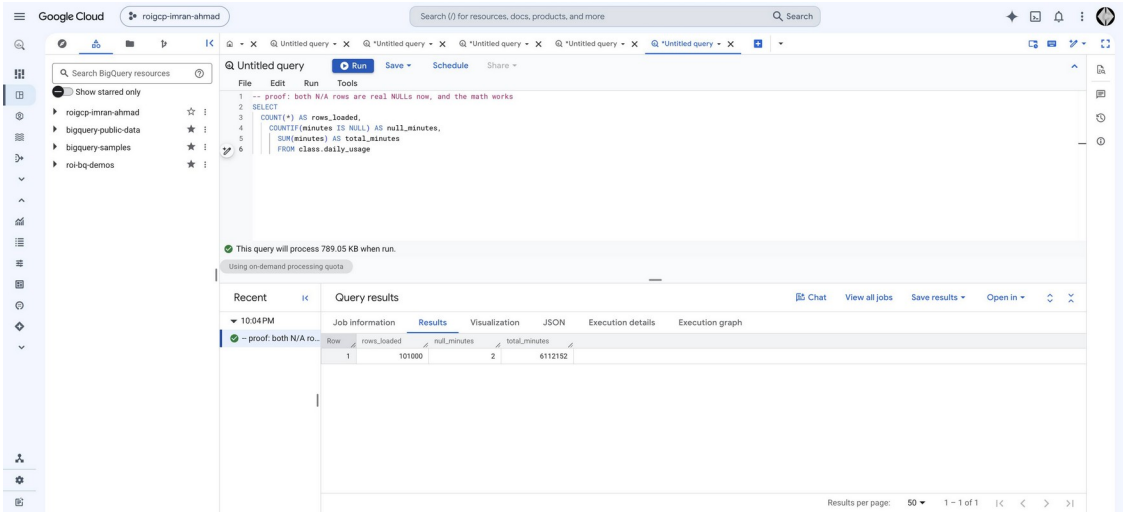


The damage surfaces later, in every query that does math:



The fix — pinned schema + declared null marker

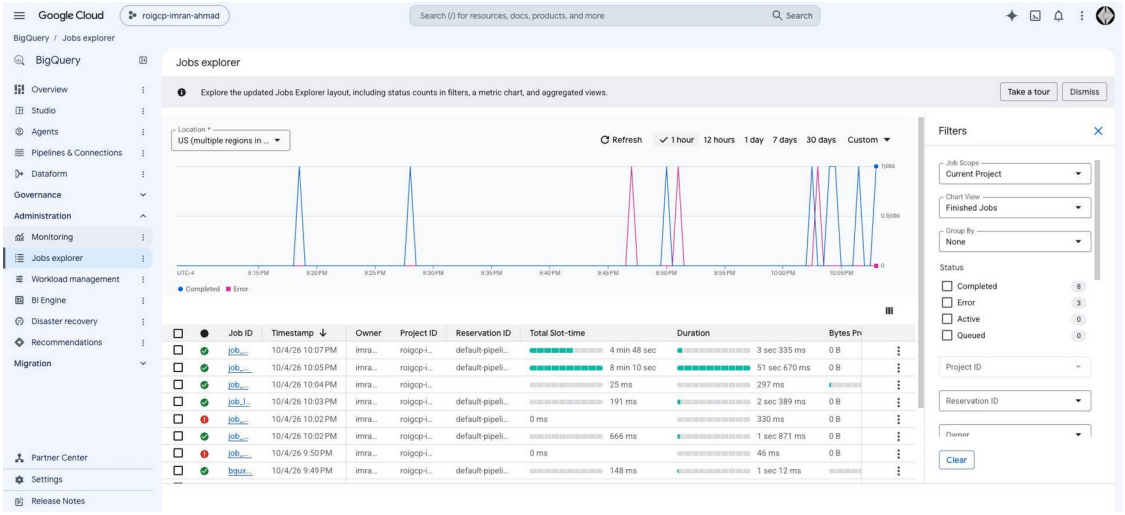
LOAD DATA with the column list pinning the schema, skip_leading_rows = 1, and null_marker = 'N/A', then the verify query: **101,000 rows, exactly 2 NULL minutes, SUM works.**



Found beyond the guide: LOAD DATA INTO with no column list does *not* just reuse the table's schema — it infers the type from the source file and validates it against the table ("Invalid schema update. Field minutes has changed type from INTEGER to STRING"). **Pin the column list on appends too.**

The format race — 60 million rows, twice

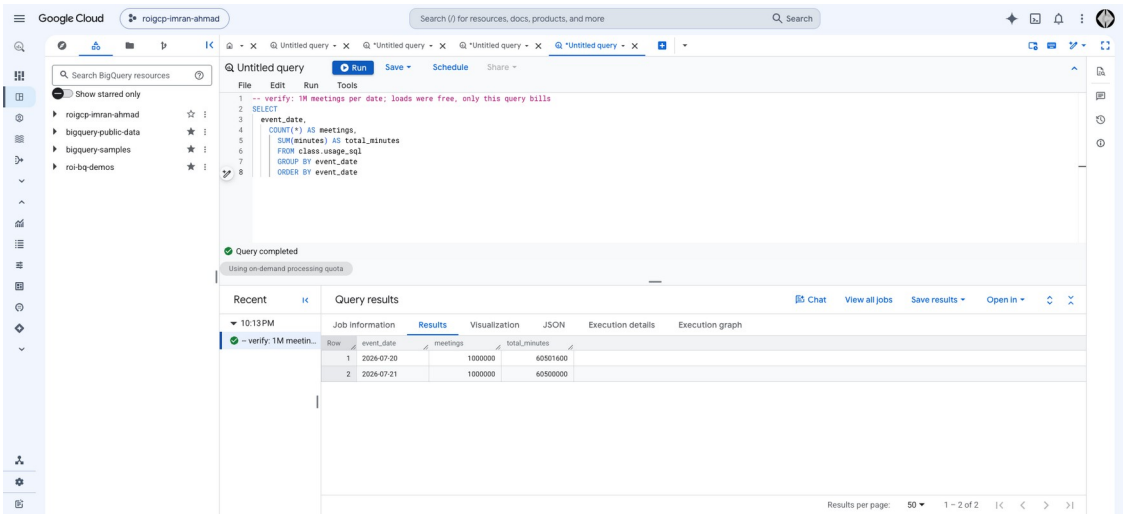
Same data, two stagings: one gzipped CSV (cannot be split — one worker alone) versus 100 Parquet files (splittable, binary, self-describing):



51.7 seconds vs 3.3 seconds — roughly 15× on the same rows, both free. (The guide's bq numbers say ~2 minutes; today's infra did it in 51.7s. The ratio is the lesson, not the absolute number.)

Everything as SQL

The activity-1 loads as pure LOAD DATA statements, verified: exactly **1,000,000 meetings per date**.



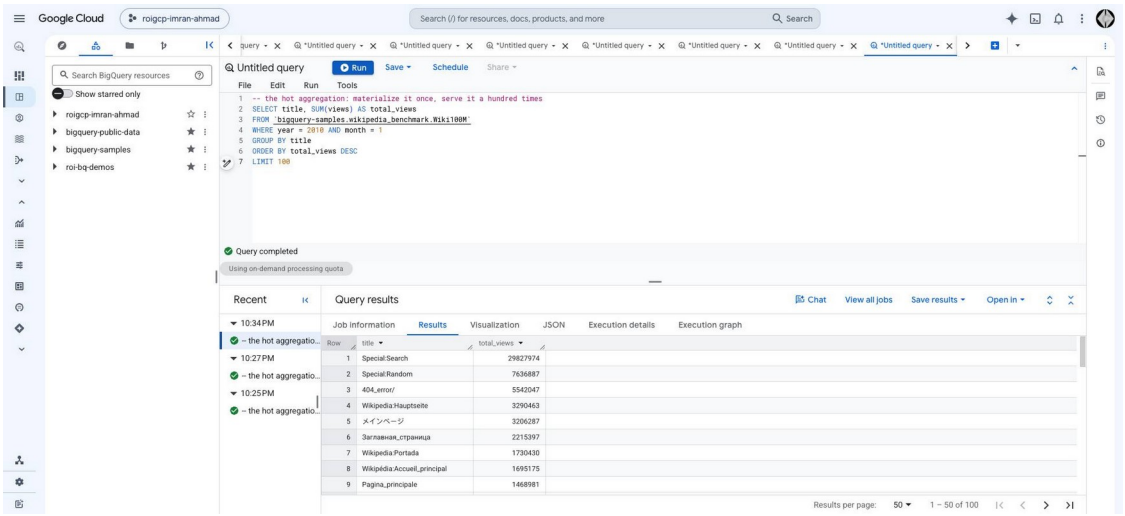
Delivery note: in the recording session, Cloud Shell's terminal rejected synthetic input, so the CLI steps were executed with their console/LOAD DATA equivalents — identical jobs on the backend. Students do the bq CLI versions in class as written.

---

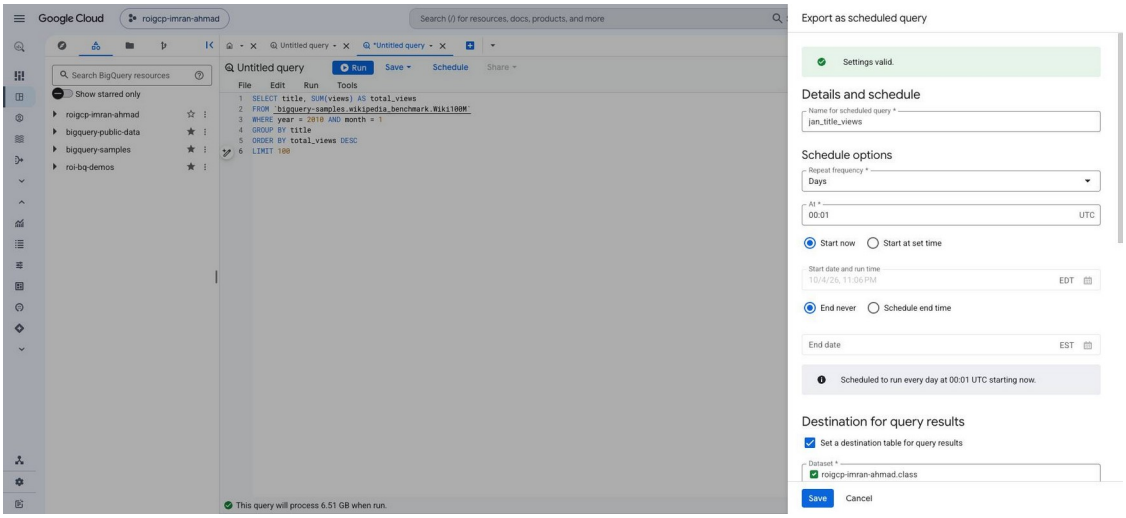
4. Day-1 Close — schedule a daily overwriting query (activity 12)

The one-sentence summary: the mechanics are one click, but the last click is a consent, not a button — and the DIN demo table wasn't where the guide said.

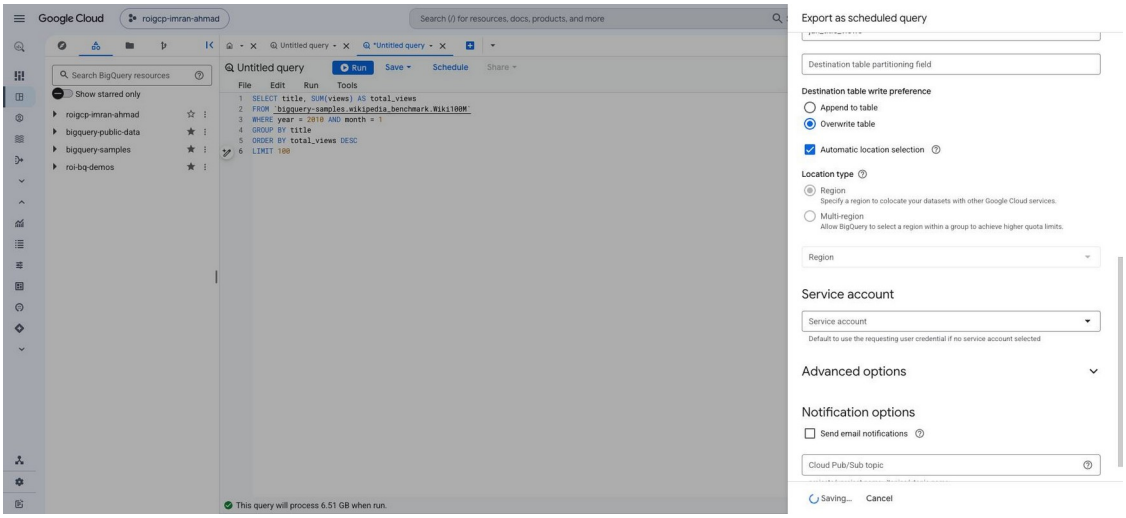
- **The DIN §29 nested table moved.** roi-bq-demos.bq_demo.nested_once was "not found," and data-to-insights / roi-datasets denied access from the instructor account. The guide already says "your instructor will confirm the shared project in class" — today the honest answer is: none of the three worked, so the recording used the public bigquery-samples.wikipedia_benchmark.Wiki100M table as the payload. Same mechanics, same cost story: the hot aggregation scans **6.51 GB**; the derived table is tiny by comparison.



- **The form's quiet requirements:** a SELECT must have a destination table (else "Form invalid"), and the daily time is HH:MM UTC — 00:01, not 12:01 AM.
- **Overwrite (WRITE_TRUNCATE) vs Append:** rebuild-from-scratch each night is idempotent — no duplicates, no merge logic. That was the choice, and it matched the guide.



- **The consent wall.** The save itself gates on the **BigQuery Data Transfer Service's OAuth consent** — the backend check literally answers hasValidCreds: false until the user clicks **AUTHORIZE** once. If the browser blocks that popup, "Saving…" spins forever with no error.



For class: treat the AUTHORIZE click as the expected final step, not a failure. If a student's save hangs: allow the popup (or run the schedule as a service account from the dropdown) and click Save again. In this recording session the popup was blocked at the browser level, so the schedule is left **fully filled and valid, one human click from created** — the mechanics are all on camera.

Across all four — the takeaways worth saying out loud

1. **Every console form has a silent-commit trap.** The IAM chip, the destination checkbox, the column list on an append — the box *looks* filled while the model is empty. Verify with the toast, the list, or the validation banner — never with your eyes alone.
2. **Docs drift faster than courses.** Three stale flows surfaced in one day (star-a-project, the lab password placeholder, the DIN table's home project). Dry-run everything the week of delivery; the live console wins any disagreement with the guide.
3. **Batch loads are free; queries bill.** That's the entire Bronze/Gold cost story in one rule — land files with loads (any front-end: console, bq, or LOAD DATA), and materialize the aggregations your analysts hammer on a schedule.
4. **The error message is not the error.** "Server closed the connection unexpectedly" meant two different setup mistakes (bad password, missing IAM grant); "Saving…" meant a missing OAuth consent. Read the failure as a class, not a sentence — then check the two usual suspects before anything exotic.